HiLCoE

School of Computer Science and Technology

INCIDENT MANAGEMENT AND MULTI-AGENCY DISPATCH SYSTEM

SENIOR PROJECT FINAL DOCUMENT

Prepared by:

ADONIAS BESUFEKAD

AMANUEL FIKREMARIAM

DARIK MESELE

HENOK BEKELE

March 2026

AI-DRIVEN GEOSPATIAL PLATFORM FOR INCIDENT MANAGEMENT AND MULTI-AGENCY DISPATCH SYSTEM FOR ADDIS ABABA CITY

Prepared by:

ADONIAS BESUFEKAD

AMANUEL FIKREMARIAM

DARIK MESELE

HENOK BEKELE

A SENIOR PROJECT DOCUMENT SUBMITTED TO THE UNDERGRADUATE PROGRAMME OFFICE IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF BACHELOR OF SCIENCE IN COMPUTER SCIENCE

Advisor:

DR. MESFIN BELACHEW

March 2026

HiLCoE

School of Computer Science and Technology

INCIDENT MANAGEMENT AND MULTI-AGENCY DISPATCH SYSTEM

Prepared by:

ADONIAS BESUFEKAD

AMANUEL FIKREMARIAM

DARIK MESELE

HENOK BEKELE

Approved By:

Advisor: **Signature:**

_____ _____

Examiner: **Signature:**

_____ _____

Acknowledgment

First and foremost, we would like to express our gratitude to **God Almighty** for granting us the determination, patience, and wisdom required to complete this senior project.

We are especially grateful to our advisor, **Dr. Mesfin Belachew**, for his professional guidance, valuable recommendations, and continuous supervision throughout the course of this project. His feedback and direction were essential in improving the technical quality, structure, and overall depth of our work.

We also extend our appreciation to the **HiLCoE School of Computer Science and Technology**, including its faculty and staff, for equipping us with the theoretical foundation, technical skills, and supportive academic environment that made this project possible.

This project would not have been successful without the strong cooperation and commitment of all team members. The shared responsibility, open communication, and collaborative effort demonstrated throughout the project played a vital role in achieving our objectives.

Finally, we would like to thank our families and friends for their encouragement, understanding, and moral support throughout this demanding academic journey.

Executive Summary

Rapid urbanization and increasing pressure on public safety services have made incident reporting and emergency response in Addis Ababa City more complex and challenging. Existing approaches for managing incidents such as traffic accidents, fires, crimes, medical emergencies, and infrastructure hazards are often fragmented and manual, leading to delayed response times, inefficient use of resources, and limited coordination among response agencies.

This senior project presents the design and implementation of an **AI-driven geospatial platform for incident management and multi-agency dispatch** tailored to the context

of Addis Ababa. The objective of the project is to develop an integrated system that supports structured citizen incident reporting, automated incident classification, geospatial visualization, and coordinated response workflows across multiple agencies.

The system was developed using an **object-oriented software engineering approach** and an **iterative vertical-slice development strategy**. It includes a web-based citizen reporting interface, agency and administrative dashboards, a backend service with role-based access control, a spatial database implemented using PostgreSQL with PostGIS, and an AI microservice for incident classification and severity scoring. Incidents are visualized on interactive GIS maps, enabling spatial queries and jurisdiction-based filtering to support operational awareness.

To address local operational realities, the platform incorporates **offline-friendly and low-bandwidth-sensitive features**, including local incident queueing and synchronization upon reconnection. Security and trust are supported through authentication, authorization, and verification mechanisms for different user roles. Real-time updates further enhance coordination during incident handling.

The implemented system demonstrates the practicality of integrating **artificial intelligence and geospatial technologies** to improve incident triage and multi-agency coordination in an urban environment. Although developed as an academic prototype, the platform provides a solid foundation for future extensions and real-world deployment.

Table of Contents

List of Figures

List of Tables

Acronyms and Abbreviations

Acronym	Definition
AI	Artificial Intelligence
AfroXLMR	Multilingual Cross-lingual Language Model for African Languages
API	Application Programming Interface
E2E	End-to-End (Testing)
EMA	Ethiopian Mapping Agency
ERD	Entity-Relationship Diagram
GIS	Geographic Information System
GPS	Global Positioning System
HDX	Humanitarian Data Exchange
HTML	Hypertext Markup Language
HTTP	Hypertext Transfer Protocol

IETF	Internet Engineering Task Force
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
NLP	Natural Language Processing
OGC	Open Geospatial Consortium
ORM	Object-Relational Mapping (e.g., Prisma)
OWASP	Open Web Application Security Project
PostGIS	Spatial database extender for PostgreSQL
PWA	Progressive Web Application
RBAC	Role-Based Access Control
REST	Representational State Transfer

RTM	Requirements Traceability Matrix
SDS	System Design Specification
SRS	Software Requirements Specification
UI	User Interface
UX	User Experience

1. Introduction

1.1 Background

Addis Ababa City has experienced rapid urban growth, increased population density, and expanding transportation networks over the past years. While this growth has contributed to economic and social development, it has also placed significant strain on public safety and emergency response services. Incidents such as traffic accidents, fires, crimes, medical emergencies, and infrastructure-related hazards occur frequently and often require timely and coordinated responses from multiple agencies.

Despite the critical nature of these incidents, existing incident reporting and response mechanisms in Addis Ababa have largely remained fragmented and manual. In many cases, incidents are reported through phone calls, informal communication channels, or agency-specific systems that operate in isolation. These approaches often result in incomplete incident information, unclear location descriptions, delayed response times, and limited coordination between responsible agencies.

Recent advancements in **Geographical Information Systems (GIS)** and **Artificial Intelligence (AI)** have demonstrated strong potential in addressing such challenges by enabling precise geospatial representation, automated incident classification, and improved decision support. By integrating these technologies into a unified platform, it becomes possible to enhance situational awareness, support efficient resource allocation, and improve overall emergency response effectiveness.

This project was undertaken to explore the application of AI and GIS technologies in the development of an integrated incident management and multi-agency dispatch system tailored to the operational context of Addis Ababa City.

1.2 Statement of the Problem

Current incident management and emergency response processes in Addis Ababa suffer from several critical limitations. One major challenge is the reliance on fragmented and agency-specific reporting channels, which limits information sharing and delays coordinated responses. Incident details are often recorded inconsistently, and location information is frequently ambiguous due to informal addressing systems, making it difficult to determine proximity to responders or agency jurisdiction.

Additionally, the absence of intelligent triage mechanisms means that incident prioritization is largely dependent on manual judgment. This can result in high-priority incidents being delayed while limited resources are allocated inefficiently. Furthermore, existing systems do not adequately support structured geospatial analysis, preventing authorities from identifying incident hotspots, temporal trends, or patterns that could inform long-term planning and prevention strategies.

Connectivity constraints and low-bandwidth conditions in certain areas further complicate system adoption, particularly for field personnel who rely on mobile access. These challenges collectively contribute to longer response times, inefficient use of emergency resources, and reduced transparency in how incidents are handled

1.3 Objectives

1.3.1 General Objective

The primary goal of this project was to design and implement GEORISE, an AI-driven geospatial platform for incident management and multi-agency dispatch tailored for the high-density urban context of Addis Ababa. This objective aimed to modernize the city's public safety infrastructure by replacing fragmented manual processes with an integrated system capable of real-time visibility and automated resource allocation [12, 13]. The project focused on leveraging a microservices architecture to ensure the system could scale independently as urban data requirements grow [10].

1.3.2 Specific Objectives

To achieve the general objective, the project focused on the systematic execution of several technical milestones. The process began with identifying and analyzing the unique functional and non-functional requirements suitable for the local administrative landscape [13]. A central milestone was the design of a modular, service-oriented architecture that supports distinct roles for citizens and emergency personnel, utilizing FastAPI to handle high-performance asynchronous workloads [6].

Furthermore, the project aimed to develop a geospatially enabled database using PostGIS to execute complex jurisdictional queries via the Simple Feature Access standard [1, 3]. In parallel, the implementation focused on integrating advanced natural language processing through the fine-tuned AfroXLMR language model to automate incident classification and severity estimation [4]. To manage operational workflows such as incident review and responder assignment, the project utilized distributed task queues like BullMQ to prevent system bottlenecks [11]. Finally, the design of a responsive citizen-facing Progressive Web App (PWA) ensured that users could report hazards with precise

coordinate capture and receive real-time status updates even in low-bandwidth environments [9].

1.4 Scope and Limitations

1.4.1 Scope

The scope of this project encompassed the development of a functional prototype that serves as a technical proof-of-concept for AI-assisted dispatch. The implementation includes a robust authentication system based on the JSON Web Token (JWT) profile to ensure secure session handling across all portals [15]. For the reporting interface, the project utilized the Leaflet library to provide an interactive GIS visualization layer for pinpointing incident locations with high accuracy [2].

Technically, the platform was built using a modern stack consisting of a Node.js runtime for the primary API gateway and a specialized AI microservice for processing multilingual narratives [10, 5]. The project specifically targeted the processing of incident data within the official administrative boundaries and sub-city structures of Addis Ababa [12]. While the architecture is designed for future extensibility, the current scope focuses on internal platform coordination and does not include live, direct integrations with national emergency short-codes or existing legacy government databases.

1.4.2 Limitations

The **GEORISE** platform is engineered as a high-fidelity functional prototype, designed to demonstrate the feasibility of an AI-driven, multilingual incident response framework for the Ethiopian market. While the system is fully operational for demonstration purposes, its current iteration is subject to the following constraints:

Technical & Architectural Limitations

- **Data Modeling and Spatial Indexing**: The current architecture utilizes the **Prisma ORM** for rapid relational data modeling. While highly efficient for the prototype's current load, production-grade scaling for high-volume, real-time spatial indexing

would require migration to native **PostGIS** extensions to optimize complex geographical queries.

- **AI Model Fidelity**: The incident classification engine is implemented as a **transformer-based proof-of-concept**. Although it successfully triages reports into categories (e.g., Police, Fire, Medical), it is not yet intended for high-stakes automated dispatching without extensive training on larger, verified real-world datasets from Addis Ababa.
- **Simulated External Integrations**: To ensure reliability during academic demonstrations, certain external dependencies—such as **SMS Gateways (OTP)** and **GPS Responder Tracking**—operate in a localized "Simulation Mode". These modules print verification codes to the system console rather than transmitting via live cellular networks to avoid service outages or quota limits.

Operational Scope

- **Geographical Focus**: The platform's GIS anchors and administrative boundaries are currently optimized for the **Addis Ababa** metropolitan area. Scaling the platform to a national level would require integrating the full **Ethiopia_AdminBoundaries.geojson** dataset and regionalized agency command nodes.
- **Linguistic Immersion**: While the system supports **English, Amharic, Oromo, and Somali**, the depth of translation is focused on critical user-facing paths (Signup, Login, and Reporting). Deep administrative settings and technical logs remain primarily in English to support developer-level diagnostics.

1.5 Methodology / Approach

The development of the AI-driven geospatial incident management and multi-agency dispatch system followed a structured yet iterative software engineering methodology. The approach combined object-oriented design principles with a hybrid vertical-slice implementation strategy, allowing core functionalities to be developed incrementally while maintaining alignment with system requirements and architectural decisions. This

methodology ensured modularity, scalability, and continuous validation of system behavior throughout the project lifecycle.

1.5.1 Requirements Analysis

The project began with a detailed requirements analysis phase aimed at understanding the operational challenges of incident reporting and emergency response coordination in Addis Ababa. This phase involved identifying key system stakeholders and defining user roles, including citizens, agency personnel, and system administrators. Functional requirements were derived by modeling core workflows such as citizen incident reporting, agency incident review, dispatch, and resolution tracking.

Non-functional requirements, including security, usability, performance, and reliability, were also identified to guide system design decisions. The outputs of this phase included a structured Software Requirements Specification (SRS), use case descriptions, and preliminary interface concepts.

1.5.2 System Design and Modeling

System design and modeling were conducted using object-oriented software engineering principles. UML diagrams were employed to represent system behavior and structure, including use case diagrams for user interactions, class diagrams for core entities, and sequence diagrams for key workflows.

A modular architecture was designed to separate concerns across major subsystems, including user management, incident processing, AI classification services, geospatial data handling, and notification mechanisms. Special consideration was given to the integration of GIS components using PostGIS and the interaction between the core backend services and the AI microservice.

1.5.3 Technology Stack and Tools

The system was implemented using modern, open-source technologies suitable for web-based and data-driven applications. The frontend was developed using React with TypeScript to provide responsive and user-friendly interfaces. Backend services were implemented using Node.js, supporting RESTful APIs and role-based access control.

PostgreSQL with the PostGIS extension was used for spatial data storage and geospatial queries. An AI microservice was developed using Python to perform incident classification and severity estimation. Supporting tools included Git for version control, Docker for environment management, and development utilities for testing and debugging.

1.5.4 Implementation Approach: Hybrid Vertical-Slice Strategy

A hybrid vertical-slice implementation strategy was adopted to incrementally deliver system functionality. Each development slice implemented a complete feature across the frontend, backend, database, and AI components. This approach enabled early validation of system workflows, reduced integration risks, and allowed continuous refinement based on testing outcomes.

Core slices included authentication and role management, citizen incident reporting, AI-based classification, GIS visualization, agency dashboards, and administrative controls. The strategy ensured that each slice resulted in a functional and testable system increment.

1.5.5 Testing and Evaluation Methods

Testing and evaluation were performed throughout the development process to ensure correctness, reliability, and usability. Unit testing was applied to key backend logic and AI service interactions, while integration testing validated communication between system components.

System-level testing focused on verifying complete workflows, such as incident submission, classification, visualization, dispatch, and resolution. Informal usability evaluation was also conducted to assess clarity and ease of use of the user interfaces. The results of testing were used to refine system behavior and improve overall stability.

1.6 Significance and Beneficiaries

The AI-driven geospatial incident management and multi-agency dispatch system developed in this project addresses critical challenges in urban emergency response by introducing structured reporting, intelligent triage, and shared situational awareness. Its significance extends beyond technical implementation, offering operational, institutional, and societal value within the context of Addis Ababa City.

1.6.1 Significance of the Project

The significance of this project can be examined across several key dimensions:

Operational Effectiveness

The platform improves incident handling by replacing fragmented and manual reporting processes with a unified digital system. Structured incident data, geospatial visualization, and coordinated agency workflows reduce ambiguity, support faster decision-making, and improve overall response coordination during emergency situations.

Technological Advancement

By integrating Artificial Intelligence and Geographical Information Systems within a single platform, the project demonstrates the practical application of modern technology to real-world urban challenges. Automated incident classification, severity estimation, and spatial analysis enhance situational awareness and provide a foundation for data-driven emergency response systems.

Institutional Coordination and Transparency

The system enables multiple response agencies to operate within a shared digital environment, reducing information silos and improving accountability. Centralized dashboards, incident timelines, and status tracking enhance transparency in how incidents are managed and resolved.

Academic and Developmental Value

As an undergraduate senior project, this work serves as a reference model for applied software engineering projects that combine AI, GIS, and distributed system design. It demonstrates the feasibility of building complex, multi-layered systems within academic constraints while addressing locally relevant problems.

1.6.2 Beneficiaries of the Project

The system benefits several stakeholder groups, each in distinct ways:

Citizens

Citizens benefit from a structured and accessible platform for reporting incidents and tracking their status. Clear reporting workflows and status visibility improve trust and encourage responsible engagement with public safety services.

Emergency Response Agencies

Agencies such as police, fire services, ambulance providers, and disaster management authorities gain access to real-time incident data, geospatial context, and coordinated workflows. This supports more efficient resource allocation and improved inter-agency collaboration.

System Administrators and City Authorities

Administrators benefit from centralized oversight, system analytics, and configurable controls for managing users, agencies, and incident data. Aggregated and spatially structured data can support planning, evaluation, and policy development.

Academic and Research Community

The project contributes to the academic community by providing a practical case study in AI-assisted geospatial systems, multi-agency coordination, and urban incident management within a developing city context.

1.6.3 Adoption Considerations and Risks

While the platform demonstrates strong technical feasibility, successful real-world adoption would depend on several non-technical factors. Organizational readiness, user training, and clear definition of agency roles are essential to ensure effective usage. Data accuracy and responsible reporting must also be addressed to prevent misuse or information overload.

Additionally, integration with existing government systems, communication infrastructure, and legal frameworks would require careful planning and coordination. These considerations highlight the importance of phased deployment, pilot testing, and institutional support in any future extension of the system.

1.7 Organization of the Document

This document is organized into several chapters. Chapter One introduces the background of the study, problem statement, objectives, scope, limitations, methodology, and significance of the project. Chapter Two presents the System and Software Requirements Specification, detailing functional and non-functional requirements, system models, and

user interfaces. Chapter Three describes the system and object-oriented design, including architectural decisions and UML models. Chapter Four discusses system implementation and testing. The document concludes with references and annexes containing the user manual and supporting materials.

2. System and Software Requirements Specification

2.1 Introduction

This chapter presents the **System and Software Requirements Specification (SRS)** for the AI-driven geospatial incident management and multi-agency dispatch system developed in this project. The SRS defines the functional and non-functional requirements of the system and serves as a reference for understanding how the platform is expected to operate under real-world conditions.

The purpose of this specification is to clearly document system capabilities, user interactions, constraints, and quality attributes such as security, usability, performance, and reliability. By formalizing these requirements, the SRS establishes a shared understanding between stakeholders and provides a foundation for the system design, implementation, and testing phases described in subsequent chapters.

This chapter also outlines the limitations of the existing incident management approach and describes the scope of the proposed solution, including system users, core functionalities, and supporting technologies. The requirements documented here ensure that the system remains aligned with the project objectives while addressing the operational challenges of incident reporting, coordination, and response within an urban environment.

2.2 Current System

Prior to the proposed system, incident reporting and emergency response coordination in Addis Ababa relied on a combination of manual procedures and agency-specific tools. Citizens typically reported incidents through phone calls, informal messages, or direct visits to nearby offices. These reports often lacked standardized formats, precise geolocation data, and consistent categorization, making it difficult for response agencies to quickly assess incident severity or urgency.

Emergency response agencies such as police, fire services, ambulance providers, and disaster management authorities operated largely in isolation. Each agency maintained its own internal workflows, records, and communication channels. When incidents required multi-agency involvement, coordination was commonly achieved through phone calls or ad-hoc communication, which increased the likelihood of delays, miscommunication, and duplicated efforts.

The existing approach provided limited support for centralized data storage or geospatial analysis. Incident locations were frequently described verbally, and historical incident data was not systematically stored in a manner that enabled trend analysis or performance evaluation. As a result, situational awareness was limited, and decision-making was primarily reactive rather than data-driven.

Additionally, current systems did not adequately account for connectivity constraints or low-bandwidth conditions, particularly for field personnel operating in dynamic environments. The absence of integrated digital platforms, automated triage mechanisms, and shared operational views significantly constrained the effectiveness and scalability of incident management efforts.

2.2.1 Challenges of the Current System

The existing approach to incident management in Addis Ababa is primarily defined by its reliance on manual, siloed processes that fail to leverage modern computational intelligence. The limitations of this system are deeply rooted in the lack of a unified technical framework, which manifests in several critical operational bottlenecks.

1. Data Fragmentation and Reporting Inefficiency Current reporting channels are highly fragmented, relying on a mix of uncoordinated verbal reports and informal messages. This lack of a centralized ingestion layer leads to a high incidence of duplicated or incomplete information. Without a structured "RECEIVED" state in a centralized database—as implemented in the GEORISE backend—agencies struggle to maintain a single source of truth for urban hazards.

2. Lack of Standardization and Geospatial Ambiguity A primary hurdle in the existing system is the inconsistency of incident formats and the ambiguity of location descriptions. Traditional reporting lacks precise geolocation data, forcing responders to rely on vague verbal landmarks. This absence of geospatial intelligence prevents agencies from performing spatial analysis or achieving real-time situational awareness. The current

manual method cannot support automated jurisdiction-based filtering or proximity-based duplicate detection—features that require the integration of a spatial database like PostGIS.

3. Decision-Support and Scalability Constraints Inter-agency coordination is currently limited by agency-specific systems that operate in isolation, with minimal real-time information sharing. Incident prioritization is entirely dependent on manual judgment, which creates significant delays during high-volume periods. Because existing workflows rely on human intervention for every triage task, they possess severe scalability constraints and cannot effectively adapt to rapid urban expansion or spikes in incident reports.

These fundamental technical gaps highlight the urgent necessity for an integrated, geospatially enabled, and intelligent platform. By replacing manual triage with AI-driven classification and GIS-based coordination, a system like GEORISE can provide the scalable framework required for a modern, coordinated emergency response in Addis Ababa.

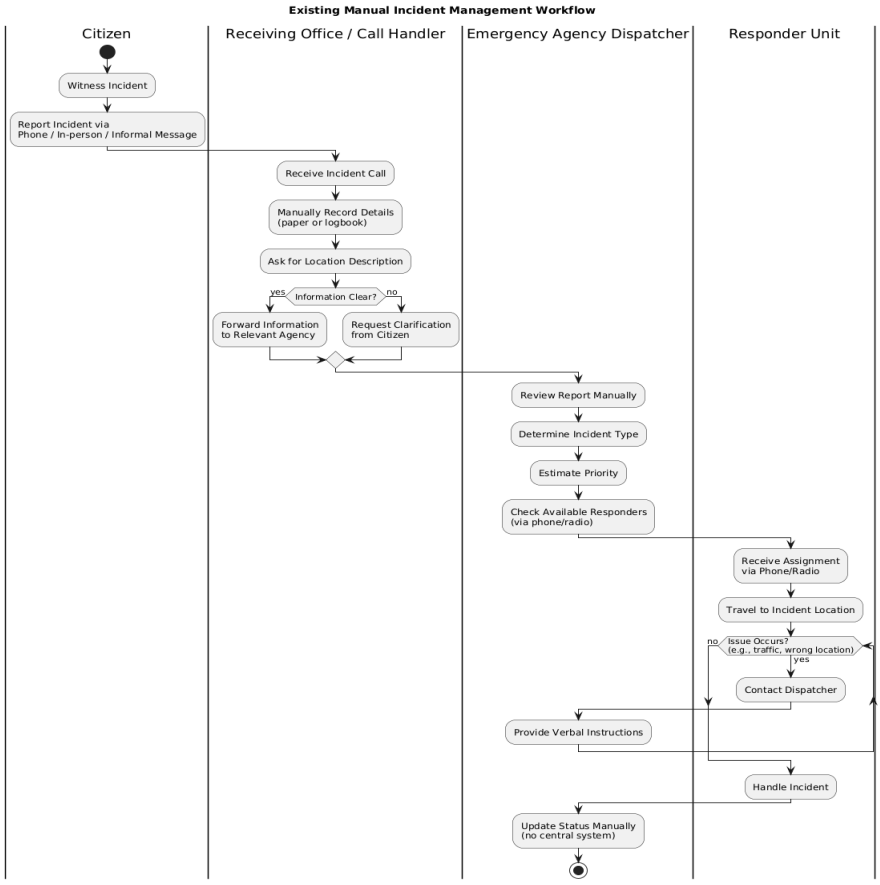

Figure 2.1: Current Manual Incident Management Workflow

2.3 Proposed System

The proposed system, **GEORISE**, replaces the fragmented and manual incident management approach with an integrated, AI-driven, and geospatially enabled digital platform. The system introduces a centralized environment where incident reporting, classification, visualization, and multi-agency coordination are handled through a unified workflow rather than isolated communication channels.

GEORISE is designed as a web-based platform that supports multiple user roles, including citizens, emergency response agencies, and system administrators. Citizens submit structured incident reports through a digital interface, while agencies access real-time dashboards to review incidents, assign responders, and track resolution progress. Artificial Intelligence is employed to assist in incident classification and severity estimation, and Geographical Information Systems (GIS) are used to provide spatial context and situational awareness.

The system architecture emphasizes modularity, role-based access control, and resilience to connectivity constraints. By combining intelligent automation with geospatial analysis and real-time communication, GEORISE addresses the key limitations of the existing system while enabling scalable, transparent, and coordinated emergency response operations.

2.3.1 System Context and Actors

The proposed system involves several actors, each interacting with the platform according to defined roles and responsibilities.

Citizen
Description: An individual who witnesses or is affected by an incident.
Key Needs: Report incidents easily, provide location information, and track incident status.

Agency User (Dispatcher / Operator)
Description: Personnel from emergency response agencies such as police, fire services, ambulance providers, or disaster management authorities.
Key Needs: Review incidents, assess priority, assign responders, update incident status, and coordinate with other agencies.

Responder Unit
Description: Field personnel responsible for handling incidents on the ground.
Key Needs: Receive assignments, access incident location details, and update resolution status.

System Administrator
Description: Manages platform configuration and oversight.

Key Needs: Manage users and agencies, monitor system health, and maintain data integrity.

2.3.2 System Modules

The GEORISE platform is engineered as a modular, service-oriented system designed to ensure high availability, technical scalability, and a clear separation of concerns across its high-latency AI and real-time geospatial layers. The proposed system is organized into the following logical modules, each responsible for a distinct phase of the incident management lifecycle.

1. User Authentication and Identity Management Module

This module serves as the security gateway for the platform, orchestrating access for citizens, agency staff, field responders, and system administrators. Beyond standard registration and login, it implements a robust identity framework utilizing JSON Web Tokens (JWT) with refresh token rotation to ensure secure, long-lived sessions while maintaining the ability to revoke access via a centralized token versioning system. Access control is enforced through a strict Role-Based Access Control (RBAC) model that programmatically restricts system capabilities based on verified identities. Furthermore, a specialized trust verification workflow allows administrators to validate citizen credentials, which in turn influences a dynamic "Trust Score" used to prioritize high-confidence reports during large-scale urban emergencies.

2. Incident Reporting and Ingestion Module

Serving as the primary interface for data collection, this module encapsulates the citizen-facing Progressive Web App (PWA) reporting layer. It utilizes a multi-step submission wizard designed to maximize data quality by structuring hazard details into predefined categories and descriptive narratives. To address the unique constraints of the Addis Ababa urban environment, the module features an offline-first reporting engine that queues incidents in local storage during network disruptions and automatically synchronizes with the backend once connectivity is restored. Geographic precision is ensured through an interactive map ingestion tool that captures high-fidelity GPS coordinates, eliminating the ambiguity associated with traditional verbal location descriptions and informal addressing systems.

3. AI Classification and Triage Module

This module operates as an intelligent microservice layer built on FastAPI to automate the initial assessment of incoming data. It leverages a fine-tuned AfroXLMR model to perform multilingual natural language processing, allowing the system to classify incident descriptions in both Amharic and English with high semantic accuracy. The module executes a comprehensive triage pipeline that includes automated severity estimation and priority scoring. To prevent false positives in high-stress scenarios, a specialized preprocessing layer applies linguistic heuristics to detect negation logic, ensuring that descriptions indicating a lack of hazard (e.g., "no fire detected") are appropriately flagged before reaching the dispatcher's queue.

4. Geospatial Intelligence and GIS Module

Acting as the core decision-support engine, this module leverages PostGIS spatial extensions to manage the platform's geographic data and jurisdictional logic. It is responsible for executing real-time spatial queries, such as jurisdiction enforcement via ST_Within functions, which automatically route incoming reports to the appropriate sub-city or woreda agency based on Addis Ababa's administrative boundaries. The module also implements a hybrid duplicate detection algorithm that identifies redundant reports by calculating geographic proximity alongside semantic similarity. For strategic oversight, it aggregates incident data to generate dynamic heatmaps and clustering visualizations, enabling authorities to identify dangerous urban patterns and incident hotspots across the city.

5. Incident Management and Dispatch Module

This module provides the primary operational interface for agency dispatchers, facilitating the transition of a report from initial ingestion to field resolution. It manages a real-time incident queue sorted by AI-derived severity and arrival time, allowing for rapid triage of critical hazards. The module orchestrates resource allocation by matching incidents to available field responders within the same spatial jurisdiction. It enforces strict state machine transitions across the incident lifecycle, ensuring that status updates—such as moving from Received to Assigned or Responding—are validated against the system's business logic to maintain an accurate and synchronized operational picture.

6. Notification and Real-time Communication Module

To maintain a continuous feedback loop between citizens and emergency services, this module manages the system's multi-channel communication bus. It utilizes WebSocket technology to push instantaneous status updates to user dashboards, providing citizens with transparency regarding the progress of their reports. For background operations, the module integrates with the Web Push API to alert agency staff of new high-priority emergencies even when the application is not actively in focus. Additionally, it supports a spatial broadcast feature, allowing administrators to push urgent safety alerts to all users located within a specific geographic radius of an active crisis.

7. Administration and System Monitoring Module

The final layer of the GEORISE architecture is the governance and monitoring module, which provides global oversight of the entire platform ecosystem. It handles administrative tasks such as onboarding new agencies, defining jurisdictional polygons, and managing system-wide configurations. A critical component of this module is the immutable audit logging system, which tracks every state transition and administrative action to ensure accountability and facilitate post-incident performance reviews. Finally, it monitors the health and performance of the AI microservice and spatial database, providing real-time telemetry on API latency and system throughput to ensure readiness during high-volume urban crises.

2.3.3 Functional Requirements

The functional requirements of the GEORISE platform define the fundamental operations and services the system must perform to meet the needs of citizens, dispatchers, field responders, and administrators. These requirements are engineered to resolve the technical gaps identified in current manual systems, specifically focusing on data standardization, geospatial accuracy, and automated decision support. By formalizing these capabilities, the platform provides a verifiable framework for system development, integration testing, and final quality assurance.

The following table outlines the twelve primary functional requirements (FR-01 to FR-12) categorized by their role in the incident lifecycle. These requirements explicitly map the user-facing features to the technical components—such as the PWA reporting wizard, PostGIS spatial queries, and the AfroXLMR classification engine—responsible for their execution.

Table 2.1: GEORISE Functional Requirements

ID	Functional Requirement	Description
FR-01	Multi-Step Incident Reporting	The system shall provide a multi-step digital interface (PWA) allowing citizens to report hazards with structured category data and multilingual text descriptions.
FR-02	Precise Geolocation Capture	The system shall allow users to specify incident locations using a map-based input (Leaflet), capturing latitude and longitude coordinates with high precision.
FR-03	Centralized Data Persistence	The system shall store all incident records, user metadata, and administrative configurations in a centralized PostgreSQL database managed through the Prisma ORM.

FR-04	Automated AI Classification	The system shall automatically categorize incident descriptions in Amharic and English using the fine-tuned AfroXLMR multilingual language model.
FR-05	Severity Estimation	The system shall analyze incident details to assign a severity level (0.0 to 1.0) and priority status to every incoming report.
FR-06	Interactive GIS Visualization	The system shall visualize incidents on an interactive map using heatmaps and clustering layers powered by PostGIS spatial indexing.
FR-07	Status Lifecycle Updates	The system shall allow agency dispatchers to review incoming incidents and update their lifecycle status (e.g., RECEIVED, ASSIGNED, RESOLVED).
FR-08	Responder Unit Assignment	The system shall support the assignment of incidents to available field responder units based on spatial proximity and agency jurisdiction.
FR-09	Multi-Agency Coordination	The platform shall allow for the escalation of incidents to support shared visibility and coordination between multiple emergency response organizations.

FR-10	Real-Time Sync & Notifications	The system shall push real-time status updates and priority alerts to relevant users via WebSockets and the Web Push API.
FR-11	Administrative Account Management	The system shall provide tools for administrators to manage user accounts, agency registrations, and urban jurisdictional boundaries (GeoJSON).
FR-12	Audit Logging and Telemetry	The system shall maintain an immutable log of all critical state transitions and administrative actions for accountability and performance analysis.

2.3.4 Non-Functional Requirements

While functional requirements define what the system does, non-functional requirements (NFRs) specify how the system should perform and the constraints under which it must operate. These quality attributes are essential for ensuring that the GEORISE platform is usable, secure, and resilient enough for urban emergency management in Addis Ababa.

The following table summarizes the core non-functional requirements, categorized by their technical impact on system architecture and user experience.

Table 2.2: GEORISE Non-Functional Requirements

Category	**ID**	**Requirement Descriptio**	**Technical Implementation**

		n	
Usability	NFR-01	Responsive Accessibility	The platform shall provide a unified user experience across mobile and desktop devices with a maximum of three clicks to initiate a report.
Performance	NFR-02	Low-Latency Triage	The system shall complete the end-to-end ingestion and AI classification of an incident within 1.5 seconds.
Security	NFR-03	Data Isolation	The system shall prevent unauthorized access to agency records via strict JWT-based role enforcement and database row-level logic.
Reliability	NFR-04	Offline Continuity	The system shall allow for hazard reporting during network outages, maintaining data integrity until sync is possible.
Scalability	NFR-05	Concurrent Handling	The platform shall support up to 100 concurrent dashboard users and 500 simultaneous reporting sessions without performance

			degradation.
Maintainability	NFR-06	Modular Architecture	The system shall utilize a microservice-oriented design to allow independent updates to the AI model without impacting the primary API.

2.3.5 Assumptions and Dependencies

The successful deployment and operation of GEORISE rely on several fundamental assumptions regarding the urban environment and the participating stakeholders. It is assumed that citizens have basic access to internet-enabled smartphones and possess the necessary literacy to interact with a digital reporting wizard. From an organizational perspective, the system depends on the willingness of emergency response agencies (e.g., Police, Fire, Medical) to adopt a shared, data-driven platform for inter-agency coordination. Technically, the platform is dependent on the high availability of the AfroXLMR AI microservice for incident triage and the consistent performance of the PostGIS spatial engine for jurisdiction-based routing. Any significant disruption to these services or a lack of stakeholder participation would directly impact the system's operational efficacy.

2.3.6 Constraints

As a senior project developed for academic evaluation, GEORISE operates under specific technical and operational constraints. The platform is implemented as a prototype intended for controlled simulations and does not currently integrate with live government databases or legacy emergency call center systems. Due to the limitations of the academic scope, external communication features—such as full SMS gateway integration for automated alerts—are simulated using standard notification APIs. Furthermore, the geographic data is constrained to the administrative boundaries of Addis Ababa city, and the AI model is

optimized for a specific set of high-priority urban hazard categories rather than a general-purpose emergency lexicon.

2.3.7 Acceptance Criteria

The GEORISE platform is considered successful if it satisfies the following comprehensive validation criteria, which bridge the gap between theoretical requirements and practical implementation:

1. **Reporting Success:** Citizens can successfully navigate the multi-step wizard to submit reports, including the capture of valid GPS coordinates.
2. **Operational Triage:** The Agency Dashboard accurately reflects incoming reports within the correct jurisdiction, displaying AI-derived categories and severity scores.
3. **Geospatial Integrity:** The GIS map successfully visualizes incident clusters and heatmaps, with jurisdictional boundaries correctly enforcing data access rules.
4. **Resilience:** The PWA demonstrates the ability to queue reports during a simulated offline state and synchronize them upon reconnection.
5. **Security & Audit:** The system enforces role boundaries as defined by the RBAC model and maintains an immutable timeline of all incident state transitions.

2.4 System Models

This section presents the system models used to describe the behavior and structure of the GEORISE platform. The models provide a visual and conceptual representation of how different actors interact with the system and how the system components are organized to support these interactions. These models are closely aligned with the functional and non-functional requirements defined earlier in this chapter and serve as a bridge between the requirements specification and the system design described in Chapter Three.

The system models include **use case diagrams** to illustrate user interactions and **component and architecture diagrams** to describe the high-level structural organization of the system.

2.4.1 Use Case Diagrams

Use case diagrams are employed to model the functional behavior of the GEORISE system from the perspective of its users and internal automation processes. Given the complexity of the platform and the diversity of stakeholders involved, the system behavior is represented using **multiple use case diagrams**, each focusing on a specific actor group or subsystem. This approach avoids visual overcrowding and allows each workflow to be expressed with sufficient detail.

The primary actors interacting with the system include **Citizens**, **Agency Dispatchers**, **Responder Units**, and **System Administrators**. In addition, GEORISE incorporates automated system behavior driven by AI and background services, which are also modeled

to reflect internal decision-making and event handling. Each use case diagram captures a coherent set of responsibilities and is directly traceable to the functional requirements defined in Section 2.3.

Citizen Incident Lifecycle Use Case Diagram

This diagram illustrates the complete workflow through which a citizen reports an incident and follows its progress. The citizen initiates the process by authenticating and submitting a structured incident report that includes location selection through a map interface and supporting details such as descriptions or media. Upon submission, the system automatically performs AI-based incident classification and duplicate detection. If a similar incident already exists, the report may be linked rather than creating a redundant record. Citizens are able to track the status of their reports and receive notifications as the incident progresses through response and resolution stages.

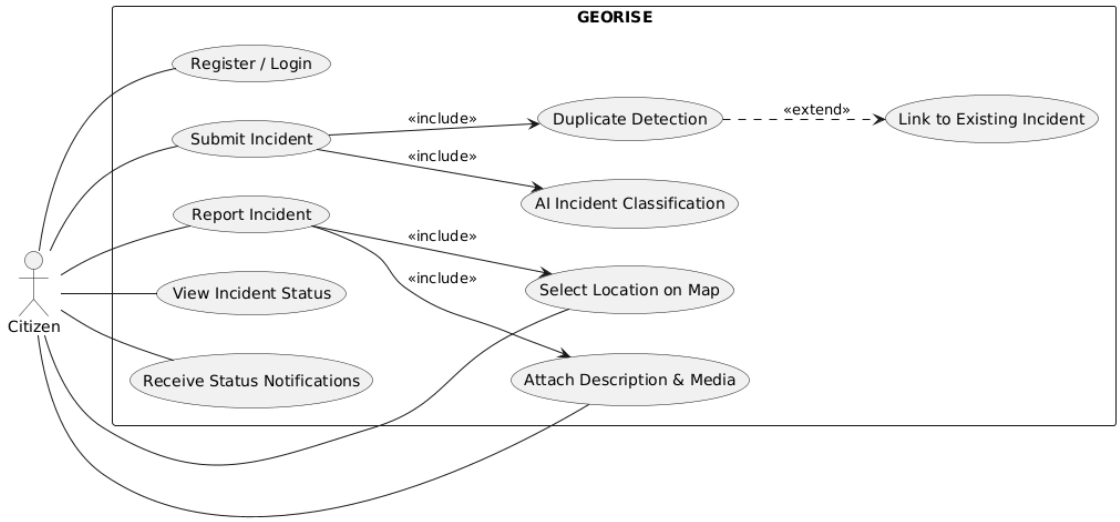

Figure 2.2: Citizen Incident Lifecycle Use Case Diagram

Agency Dispatch and Coordination Use Case Diagram

This diagram represents the core operational workflow for emergency response agencies. Agency dispatchers interact with the system to review incoming incidents, visualize them on a geospatial map, and evaluate AI-generated classifications and severity scores. Based on this information, dispatchers may adjust priorities, assign responder units, merge duplicate incidents, or escalate incidents that require multi-agency coordination. The diagram also captures ongoing incident status updates and closure activities, reflecting the full lifecycle of incident management from an agency perspective.

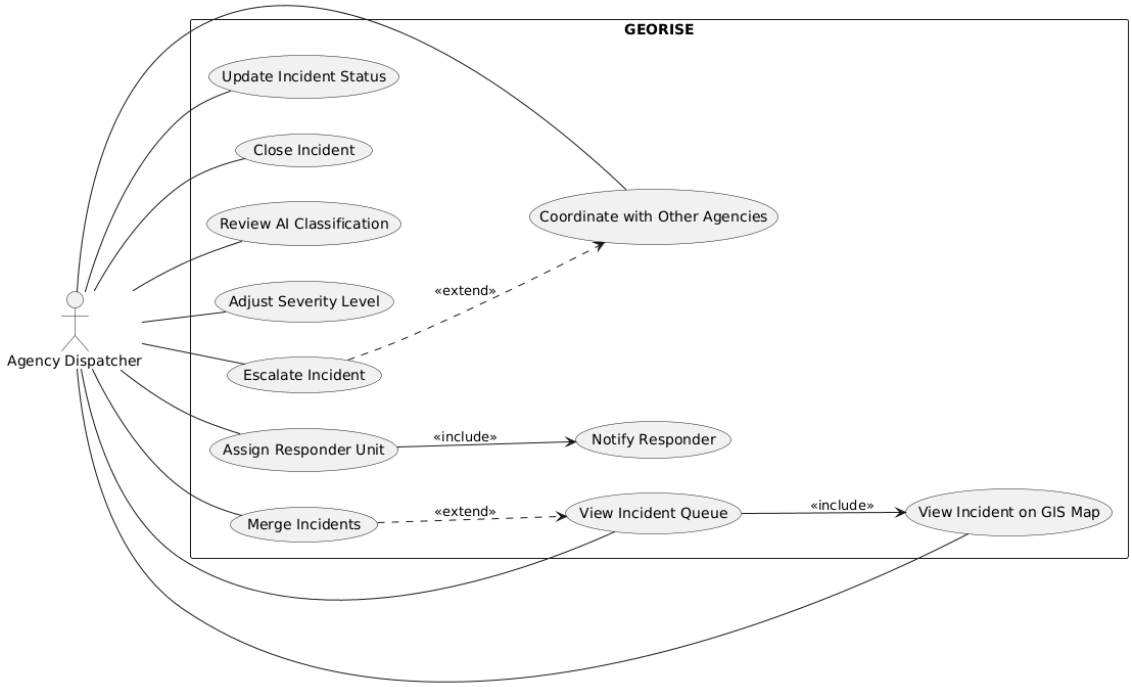

Figure 2.3: Agency Dispatch and Coordination Use Case Diagram

Responder Field Operations Use Case Diagram

The responder field operations diagram focuses on the workflow of responder units assigned to incidents. Responders receive assignments through the system, review incident details, and navigate to the incident location using provided spatial information. During response operations, responders can update incident progress, request additional support if needed, and report final resolution outcomes. This use case model highlights the real-time interaction between field personnel and the central system.

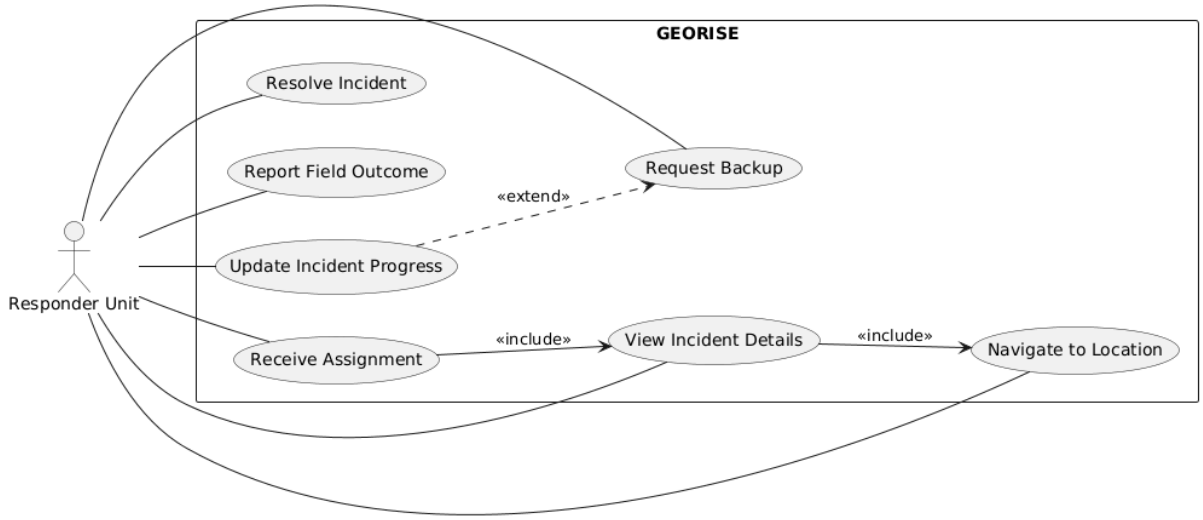

Figure 2.4: Responder Field Operations Use Case Diagram

System Administration and Control Use Case Diagram

This diagram captures the responsibilities of system administrators responsible for platform governance and oversight. Administrative functions include managing users and agencies, verifying citizen accounts, defining agency jurisdictions, and configuring system-level settings. The diagram also models advanced control features such as broadcasting alerts, activating crisis mode to restrict low-priority reporting, monitoring system health, and reviewing audit logs. These use cases ensure accountability, trust management, and operational continuity across the platform.

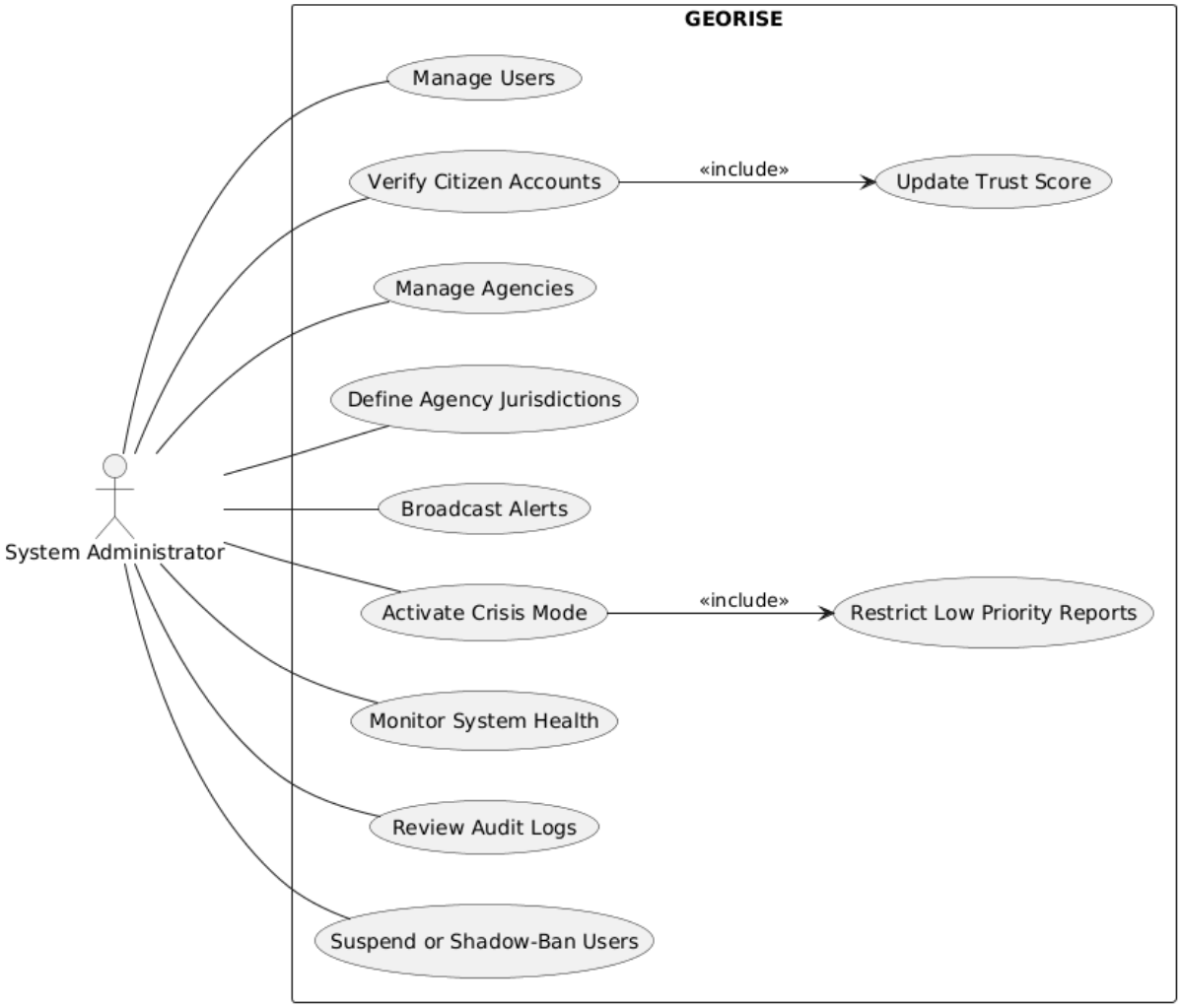

Figure 2.5: System Administration and Control Use Case Diagram

AI and System Automation Use Case Diagram

In addition to human actors, GEORISE relies on automated system behavior to support intelligent decision-making and scalability. This diagram models internal system processes such as AI-based incident classification, severity scoring, duplicate detection, dispatch recommendations, notification triggering, and audit logging. Representing these behaviors explicitly emphasizes the role of automation in reducing manual workload and improving response efficiency.

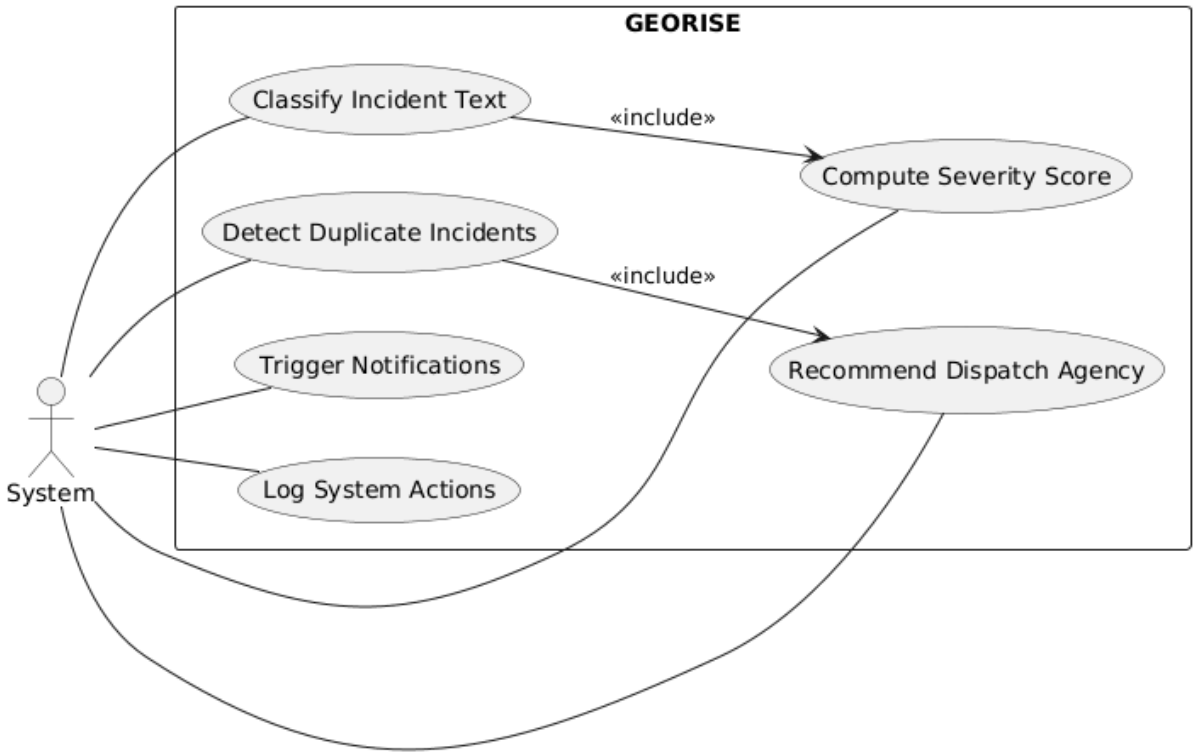

Figure 2.6: AI and System Automation Use Case Diagram

Collectively, these use case diagrams provide a comprehensive view of how GEORISE supports end-to-end incident reporting, analysis, coordination, and resolution. By separating workflows by actor and subsystem, the models clearly demonstrate system responsibilities and interactions while maintaining readability and traceability to system requirements.

2.4.2 Component and Architecture Diagram

This section presents the component and architecture models used to describe the structural organization of the GEORISE system. These models illustrate how the major software components are grouped into logical layers and how they interact to support incident reporting, analysis, dispatch, and resolution workflows. The architecture emphasizes modularity, separation of concerns, and scalability to ensure maintainability and future extensibility.

To improve clarity, the system structure is represented using two complementary diagrams: a **high-level architecture diagram** and a **detailed component diagram**. Together, these views provide both an overall understanding of the system and insight into the responsibilities of individual components.

High-Level System Architecture

The high-level architecture diagram provides an overview of the main layers and services that make up the GEORISE platform. The system is organized into five primary layers: client layer, application layer, AI layer, data layer, and external services.

The **client layer** consists of web-based interfaces for citizens, agency users, responders, and system administrators. These clients communicate with the backend through a centralized API, ensuring consistent access control and request handling.

The **application layer** hosts the core backend services responsible for authentication and role-based access control, incident management, dispatch coordination, and notification handling. This layer orchestrates system workflows and serves as the primary integration point between user interfaces, AI services, and data storage.

The **AI layer** contains services dedicated to automated incident processing, including incident classification, severity scoring, and duplicate detection. These services operate independently from the core application logic and are invoked as needed to support intelligent decision-making.

The **data layer** is responsible for persistent storage of system data. A relational database is used to store incident records, user data, and audit logs, while spatial extensions support geospatial queries and map-based visualization.

The **external services layer** includes third-party services such as mapping and routing tools and notification gateways. These services are accessed through well-defined interfaces to reduce coupling and manage external dependencies.

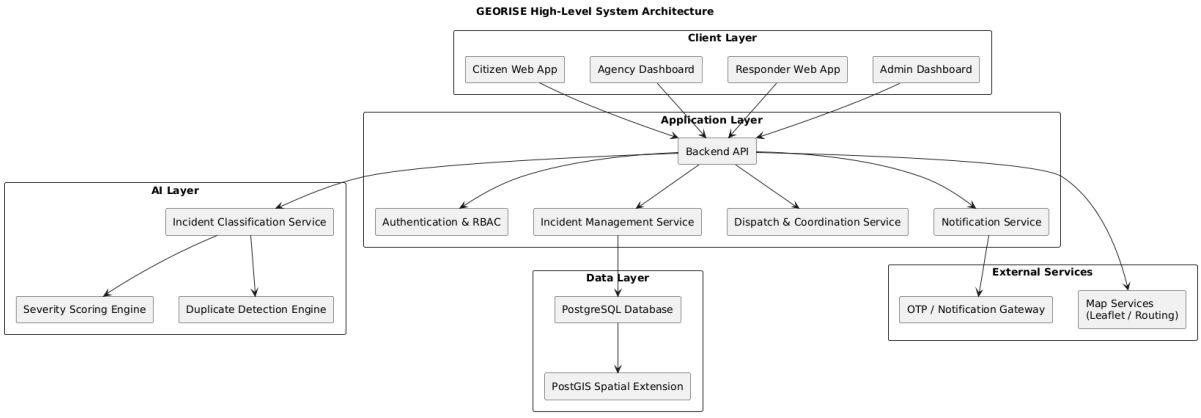

Figure 2.7: Illustrates the high-level component and architecture overview of the GEORISE system.

Detailed Component Diagram

The detailed component diagram expands on the high-level view by presenting the internal logical components of the system and their interactions. This diagram focuses on how responsibilities are distributed within each layer and how data flows between components.

Frontend components include interfaces for incident reporting, GIS visualization, agency operations, responder assignment, and administrative control. These components interact exclusively with the backend through an API gateway, which centralizes request routing and security enforcement.

Within the backend, individual controllers handle specific domains such as authentication, incident processing, dispatch coordination, administrative operations, and notifications. This separation of concerns simplifies maintenance and allows components to evolve independently.

The AI processing components form a structured pipeline that includes text preprocessing, incident classification, severity prediction, and similarity matching. This pipeline supports automated analysis while remaining loosely coupled to the main application logic.

The persistence layer includes repositories for incident data, user information, audit logs, and spatial data. This organization ensures clear ownership of data access responsibilities and supports efficient querying and auditing.

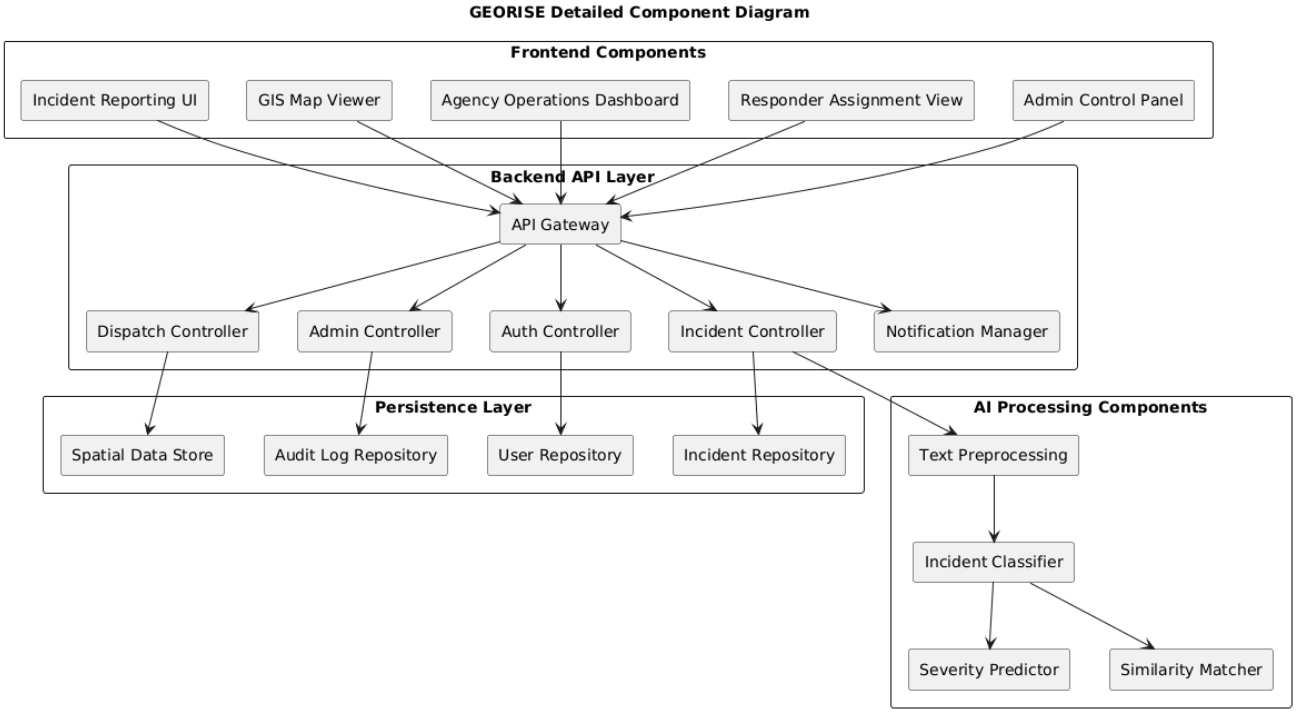

Figure 2.8: Presents the detailed component diagram of the GEORISE system.

Architectural Rationale

The layered and component-based architecture of GEORISE was chosen to support scalability, security, and ease of maintenance. By separating user interfaces, core application logic, AI processing, and data management, the system minimizes coupling and enables independent development and testing of components. This architectural approach also supports future enhancements, such as integration with additional agencies, advanced analytics, or expanded AI capabilities.

With the system behavior and structure clearly defined through use case and component models, the next chapter presents the **System Design Specification**, detailing architectural decisions, data models, and implementation strategies.

2.5 External Interface Requirements

This section defines the external interfaces required by the GEORISE system to support mapping, artificial intelligence processing, authentication, and communication. These interfaces enable integration with third-party services and internal subsystems while maintaining security, reliability, and modularity. The technical characteristics and protocols for these interactions are summarized in Table 2.3.

Table 2.3: GEORISE External Interface Requirements

Interface	Details
Map Services (Leaflet / Routing API)	**Purpose:** Provides interactive map visualization, location selection, and optional routing support for responders. **Protocol:** HTTPS REST (map tiles and routing requests). **Notes:** Used for incident location input, GIS visualization, and responder navigation; subject to rate limits.
AI Classification Service	**Purpose:** Performs incident type classification, severity scoring, and text similarity analysis. **Protocol:** HTTP REST (POST requests with JSON payload). **Notes:** Supports Amharic and English text; invoked asynchronously by backend services via BullMQ.

OTP / Notification Gateway	**Purpose:** Sends verification codes and system notifications. **Protocol:** HTTPS REST. **Notes:** SMS delivery may be simulated or partially implemented during the academic phase using external provider APIs.
Authentication Service	**Purpose:** Manages user authentication and session handling. **Protocol:** HTTPS REST with token-based authentication (JWT). **Notes:** Enforces role-based access control across all system interfaces and provides token rotation logic.
Internal REST API	**Purpose:** Primary interface between client applications (Citizen/Agency portals) and backend services. **Protocol:** HTTPS REST. **Notes:** Secured with RBAC, supports all core system operations including incident management, spatial updates, and administration.

2.6 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) ensures that each functional requirement identified in Section 2.3.3 is mapped to its corresponding system components and implementation status. This matrix supports verification, validation, and accountability throughout the development process, allowing developers and stakeholders to track the realization of specific features from requirement to code. The mapping of the core GEORISE requirements is provided in Table 2.4.

Table 2.4: GEORISE Requirements Traceability Matrix

Requirement ID & Summary	Technical Details	Status
FR-01: Incident Reporting	**Modules:** Incident Reporting Module, GIS Module **Components:** `ReportIncidentWizard.tsx`, `incident.controller.ts`	Complete
FR-02: Location Selection	**Modules:** Geospatial Management Module, Spatial Data Store **Components:** `LocationStep.tsx`, PostGIS `ST_MakePoint` logic	Complete
FR-03: AI Classification	**Modules:** AI Classification Module **Components:** FastAPI `main.py`, `AfroXLMR` inference engine	Complete
FR-04: Severity	**Modules:** AI Classification Module	Complete

Scoring	**Components:** `severity.py` utility, NLP sentiment analysis	
FR-05: Incident Visualization	**Modules:** Geospatial Management Module **Components:** `AgencyMap.tsx`, `HeatLayer.tsx`, `ClusterLayer.tsx`	Complete
FR-06: Assignment & Dispatch	**Modules:** Incident Management and Dispatch Module **Components:** `dispatch.service.ts`, `AgencyDashboard.tsx`	Complete
FR-07: Multi-Agency Coordination	**Modules:** Incident Management, Notification Module **Components:** `IncidentEscalation` logic, `sharedWith` schema fields	Partial
FR-08: User & Role Management	**Modules:** Authentication and Access Control Module **Components:** `auth.ts` middleware, Prisma RBAC enums	Complete
FR-09: Audit Logging	**Modules:** Administration and Monitoring Module	Partial

	Components: `audit.ts` middleware, `ClassificationAudit` table	
FR-10: System Notifications	**Modules:** Notification and Communication Module **Components:** `Socket.io` integration, `NotificationBell.tsx`	Partial
FR-11: Admin Management	**Modules:** Administration and Monitoring Module **Components:** `AdminDashboard.tsx`, `SystemStatusPage.tsx`	Complete
FR-12: Offline Continuity	**Modules:** Incident Reporting Module **Components:** `incidentQueue.ts`, `sw.ts` (Service Worker)	Complete

3. System Design Specification & Object Design Document

3.1 Introduction

This chapter presents the **System Design Specification (SDS)** and **Object Design Document (ODD)** for the GEORISE platform. Based on the requirements specified in Chapter Two, this chapter defines how the system is architected, decomposed, and structured at both system and object levels. The design focuses on scalability, modularity,

reliability, and real-time coordination, which are essential for an AI-driven geospatial incident response system.

The chapter is divided into two major sections:

- **System Design Document (SDD)**, describing architectural styles, subsystem decomposition, deployment considerations, and data design.
- **Object Design Document (ODD)**, detailing class structures, interactions, state management, and internal logic.

3.2 System Design Document (SDD)

3.2.1 Architectural Style (Layered & Service-Oriented)

GEORISE adopts a hybrid layered and service-oriented architectural model, wherein system functionality is distributed across a centralized backend application and specialized microservices. This approach ensures that computationally intensive tasks do not impede core transactional workflows, maintaining high performance during real-time incident response. The architecture is primarily organized into distinct tiers with clearly separated responsibilities, utilizing a microservices strategy to allow for independent scaling and maintenance as urban data requirements evolve [10].

Client Layer

The client layer is comprised of web-based Progressive Web Applications (PWAs) developed using the React framework [9]. These applications are role-aware, dynamically presenting functionality tailored to citizens, agency dispatchers, field responders, and system administrators. The reporting interface integrates the Leaflet library [2] to facilitate precise map-based location selection, while real-time state synchronization is managed through bi-directional WebSocket channels. To address the infrastructure constraints of the local environment, the client layer implements an offline-first strategy that allows incident reports and responder telemetry to be queued in local browser storage and synchronized automatically upon the restoration of network connectivity.

Application (API) Layer

The application layer serves as the primary orchestrator, implemented as a Node.js and TypeScript backend [10]. This layer exposes RESTful APIs that coordinate all system operations, including authentication, incident lifecycle management, and inter-agency communication. Security is enforced through a robust implementation of the JSON Web Token (JWT) profile [15], which facilitates secure session handling and fine-grained Role-Based Access Control (RBAC). The backend is designed to be stateless and follows modern API security standards to mitigate common risks such as unauthorized data exposure [14]. Furthermore, it utilizes distributed task queues like BullMQ to decouple the ingestion of reports from high-latency processing tasks [11].

AI and Decision Support Services

To automate the triage process, GEORISE incorporates independent AI processing microservices built on the FastAPI framework [6]. These services are responsible for analyzing incident narratives in both Amharic and English using the fine-tuned AfroXLMR multilingual language model [4, 5]. By processing descriptions through this transformer-based pipeline, the system can automatically produce classification labels, estimate severity levels, and detect potential duplicate reports based on semantic similarity [7]. This decoupling ensures that the time-intensive inference phase of the AI model does not block essential system operations like incident submission or responder dispatch.

Geospatial and Mapping Services

Geospatial intelligence is supported through a combination of a spatially enabled relational database and client-side visualization tools. The system utilizes PostGIS [1] to handle complex spatial operations, including jurisdiction enforcement and proximity-based duplicate detection. All spatial queries are executed in compliance with the Simple Feature Access (SFA) standard [3], ensuring interoperability and accuracy. The system consumes administrative boundary datasets specifically curated for the sub-city structures of Addis Ababa [12] and integrates these with common operational datasets [13] to provide dispatchers with high-fidelity situational awareness through interactive map layers [2].

Data Access and Persistence Layer

The persistence layer is built on a PostgreSQL database extended with PostGIS for the reliable storage of both relational metadata and geometry data types [1]. This layer manages the canonical records for incidents, users, and jurisdictional polygons. Data access is mediated through the Prisma ORM [8], which ensures schema consistency and provides a secure interface for executing the raw SQL required for advanced spatial analytics. By abstracting the database interactions, the platform maintains a clean separation between the physical storage layer and the business logic of the application layer.

Architectural Rationale

The selection of a layered and service-oriented approach provides the GEORISE platform with the resilience required for emergency management. By isolating AI and geospatial processing from the core business logic, the system maintains a responsive interface even during periods of high computational load. This modularity supports future expansions, such as the integration of predictive analytics or additional regional agencies, without introducing technical fragility. In practice, this design ensures that the system operates reliably in environments with intermittent connectivity, providing a stable foundation for modernizing public safety infrastructure in Ethiopia.

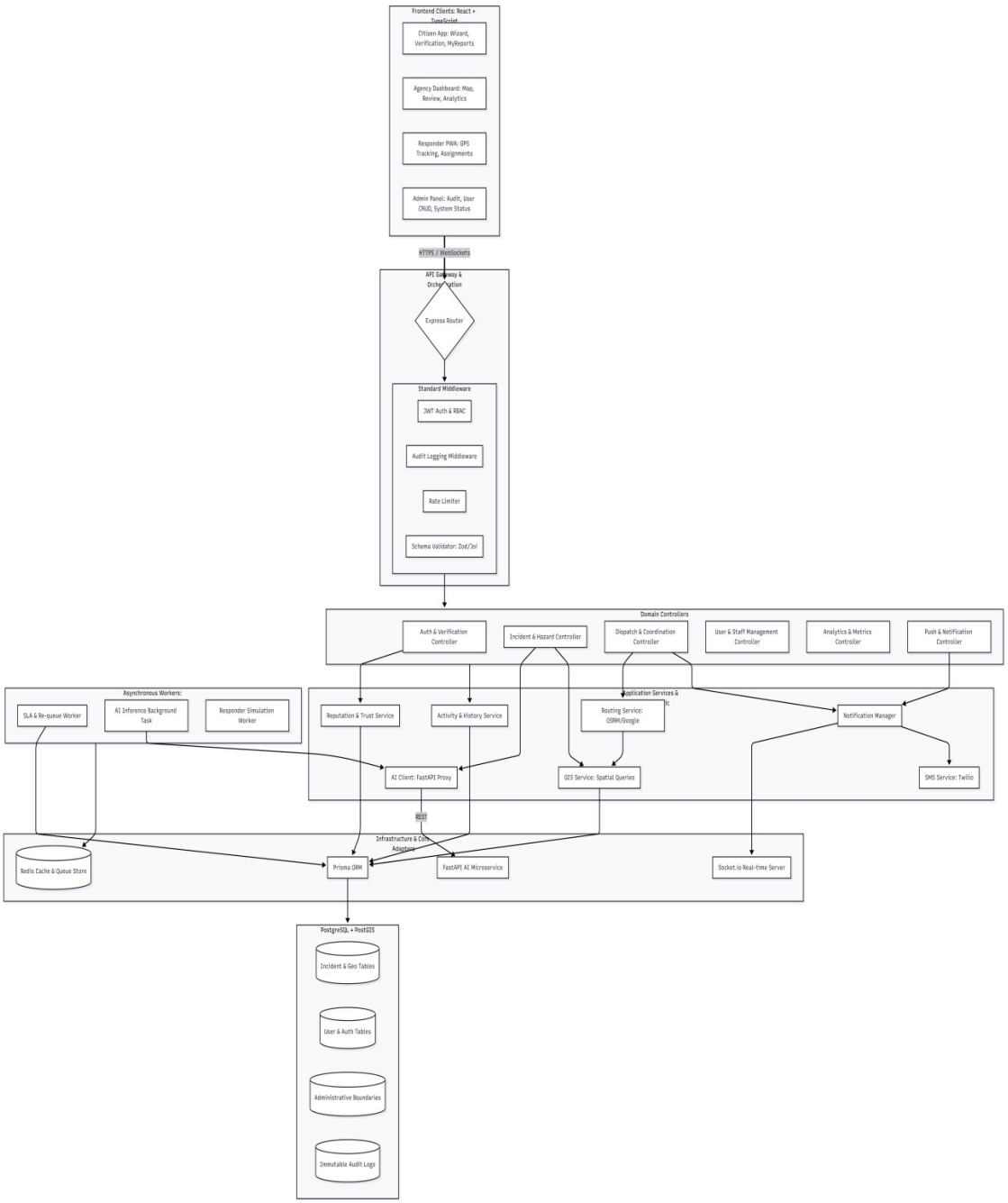

Figure 3.1: Detailed Component Interaction Diagram showing data and control flow across layers

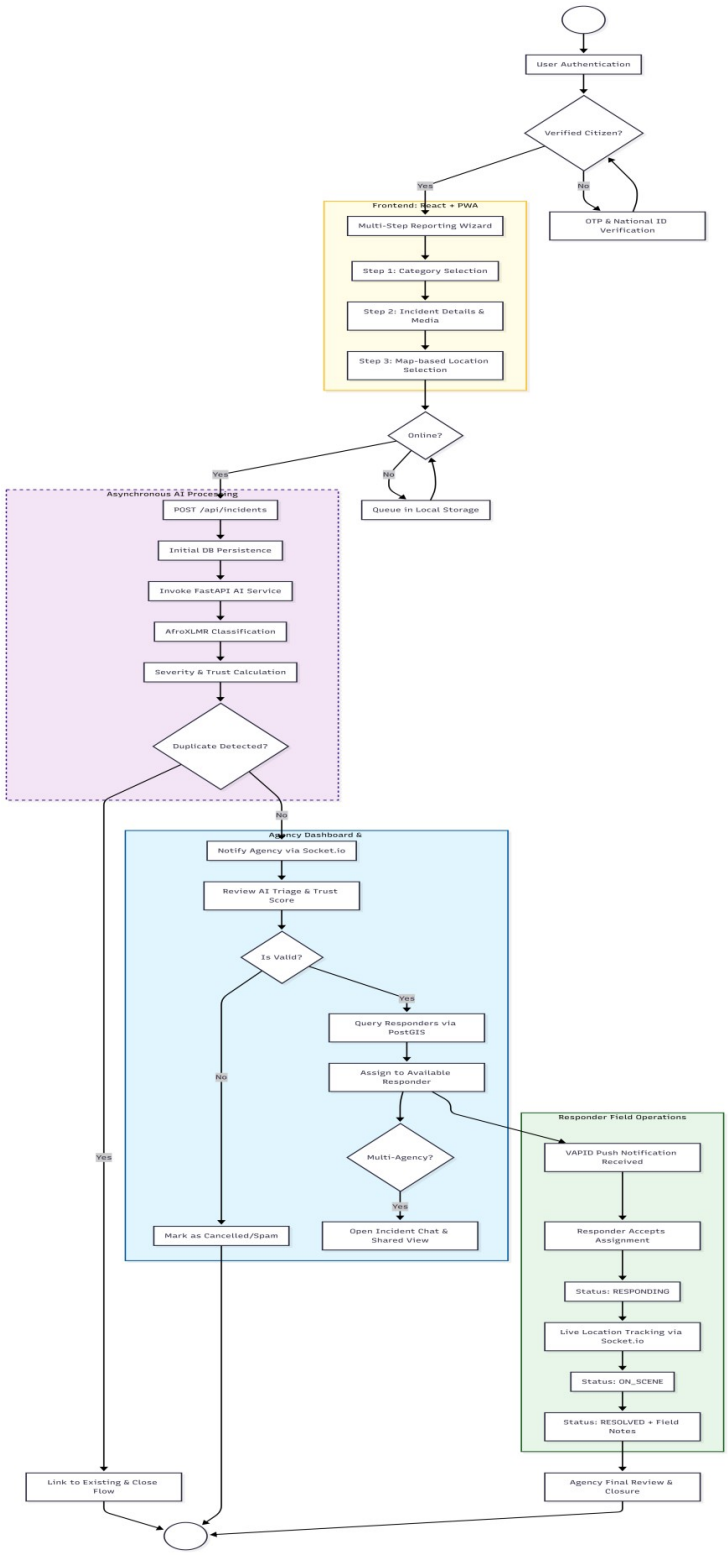

Figure 3.2: Incident Lifecycle Activity Diagram from citizen report submission to resolution.

3.2.2 High-Level System Architecture

The high-level architecture defines the interaction between client applications, backend services, AI processing units, and data storage components.

Major architectural elements include:

- Web-based client interfaces for citizens, agencies, responders, and administrators
- Centralized backend API layer
- AI-powered incident analysis services
- Geospatial processing and mapping services
- Relational database with spatial extensions

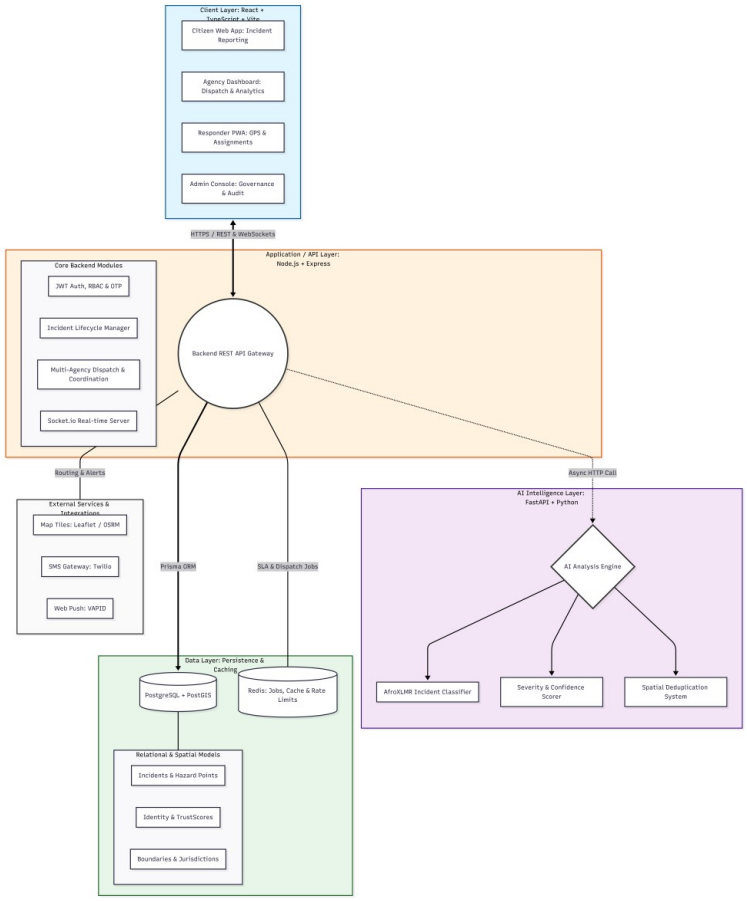

Figure 3.3 High-Level Architecture Diagram of the GEORISE System

3.2.3 Subsystem Decomposition

The GEORISE system is decomposed into a set of bounded subsystems, each responsible for a clearly defined domain of functionality. Every subsystem exposes explicit integration interfaces—primarily synchronous REST APIs and real-time messaging channels—and

owns the data or transient artifacts required for its operation. This decomposition enables independent evolution of subsystems, targeted scaling of high-load components, and clear operational ownership.

The design also reflects the realities of an emergency response environment, where responsiveness, fault isolation, and reliability are critical. Computationally intensive tasks such as AI inference are isolated from core transactional workflows, while real-time communication is handled through dedicated channels. The detailed breakdown of these subsystems is provided in Table 3.1.

Table 3.1: Subsystem Decomposition and Responsibilities

Subsystem	**Responsibilities**	**Interfaces**	**Owned Artifacts**
Client Applications (Frontend)	User interfaces for citizen incident reporting, agency dispatch dashboards, responder field operations, and administrative control; map-based interaction for location selection and visualization; offline-first data capture and synchronization; real-time updates via sockets.	HTTPS REST to `/api/*`; WebSocket channels for live updates and notifications; browser storage (IndexedDB/Cache API) for offline queues.	Transient UI state, cached map tiles, offline incident and location queues.

API / Application Backend	Central orchestration of system workflows; validation and normalization of incoming data; incident lifecycle management; enforcement of authentication, RBAC, and jurisdiction rules; coordination between AI services, GIS services, and persistence layer; exposure of real-time events.	REST endpoints (`/api/incidents`, `/api/dispatch`, `/api/admin`, etc.); WebSocket endpoints for real-time updates; internal service calls to AI processing services via BullMQ.	Canonical domain records including Incidents, Users, Agencies, Responder Units, Assignments, and Notifications.
AI Analysis Services	Automated processing of incident descriptions, including text preprocessing, incident classification (AfroXLMR), severity scoring, and duplicate	HTTP REST (POST requests with JSON payloads containing incident data).	Ephemeral inference results and confidence scores; no durable persistence of core domain data.

	detection using text similarity and spatial context.		
GIS & Spatial Services	Geospatial processing such as proximity queries, jurisdiction enforcement (`ST_Within`), incident clustering, and spatial indexing; support for map visualization and responder tracking.	Internal database queries via PostGIS spatial extensions; external map and routing APIs (Leaflet/OSRM) when required.	Spatial indexes (GIST), geometry data, and administrative boundary polygons (GeoJSON).
Notification & Real-Time Services	Generation and delivery of in-app notifications; real-time event fan-out to connected clients; fallback delivery via SMS or other channels when applicable.	WebSocket channels (Socket.io); REST endpoints (`/api/notifications`); outbound connectors to notification gateways.	Notification records, delivery status, and acknowledgment metadata.
Authentication & Trust	User authentication, session lifecycle	`/api/auth/*`; token validation hooks for REST and	User accounts, roles, trust

Management	management (JWT), role-based access control (RBAC), trust scoring, and account verification; enforcement of security policies across all subsystems.	WebSocket connections.	scores, and hashed credentials.
Audit & Compliance	Immutable logging of critical system and administrative actions; support for forensic audits, system reviews, and compliance reporting.	`/api/audit/*`; export utilities for administrative review.	Append-only audit event records and system logs.
Analytics & Reporting	Aggregation of historical incident data; generation of KPIs, heatmaps, response-time metrics, and trend analysis for agencies and administrators.	`/api/analytics`; background processing/CRON jobs for materialized view updates.	Aggregated metrics, materialized views, and reporting snapshots.

Integration and Orchestration Patterns

Synchronous user-driven workflows rely on **HTTPS REST communication** between client applications and the API backend. When a citizen submits an incident report, the backend immediately validates the request, persists the incident, and invokes AI analysis services to generate classification and severity recommendations. These results are returned asynchronously and surfaced to agency users for review.

Real-time coordination and updates are handled through **WebSocket-based communication**, enabling live incident status updates, responder tracking, and notification delivery without repeated polling.

Asynchronous processing is applied where appropriate, such as analytics aggregation and notification fan-out, ensuring that non-critical background tasks do not block time-sensitive operations. Data ownership is explicit: the API backend owns all durable persistence, while AI services remain stateless to support scalability and fault isolation.

3.2.4 Deployment Architecture

GEORISE is designed for **cloud-ready deployment** with containerized services.

Deployment characteristics:

- Stateless backend services
- Centralized relational database
- External APIs accessed through secured gateways
- Support for horizontal scaling

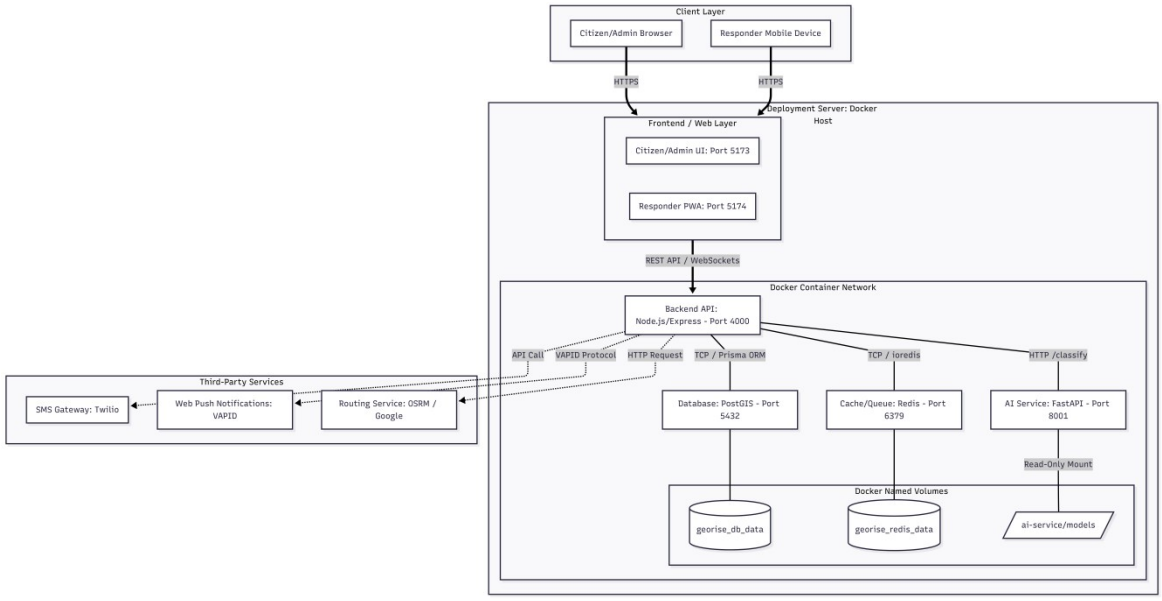

Figure 3.4: Deployment Architecture Diagram

3.2.5 Data Design and Persistence

The system uses a relational database enhanced with spatial capabilities.

Stored data includes:

- Incident records
- User and agency information
- Response assignments
- Notifications and audit logs
- Historical analytics data

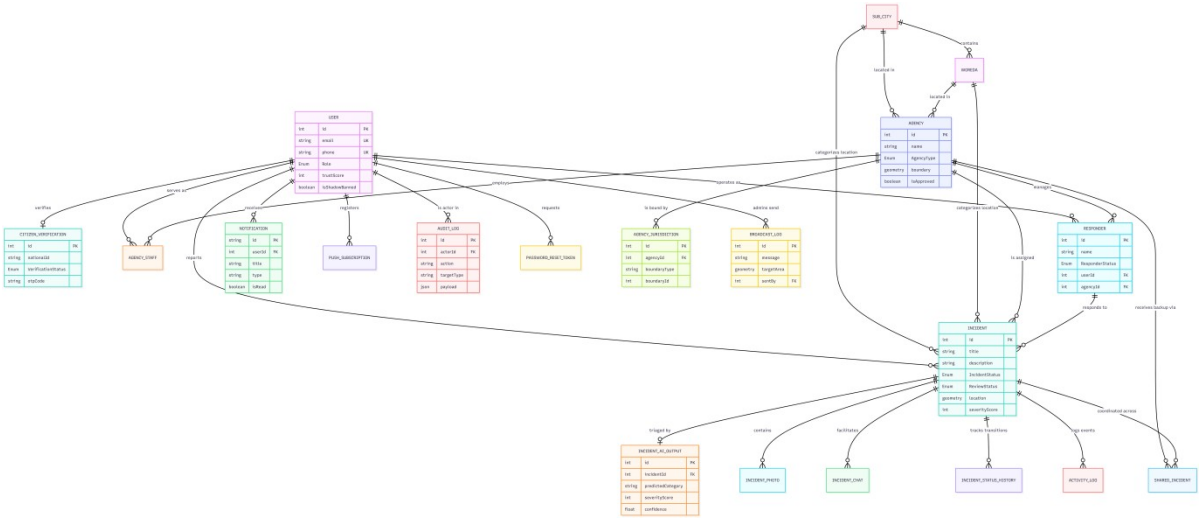

Figure 3.5: Entity Relationship Diagram (ERD)

3.2.6 Security Design

Security considerations include:

- JWT-based authentication
- Role-based access control (RBAC)
- Input validation and rate limiting
- Secure handling of sensitive data
- Comprehensive audit logging

3.2.7 Design Constraints and Trade-offs

Design decisions were influenced by:

- Academic time constraints
- Availability of external APIs
- Computational limitations for AI inference
- Infrastructure cost considerations

3.3 Object Design Document (ODD)

The Object Design Document defines the internal logical structure of the GEORISE system, bridging the gap between high-level architectural subsystems and the concrete implementation. It focuses on the organization of code into logical packages and the definition of core classes that manage data and behavior.

3.3.1 Package and Module Structure

The backend system is organized into bounded contexts represented as packages. This modularity ensures that changes to specific domains—such as the AI inference logic or

spatial operations—do not cause unintended regressions in the core identity or incident management logic.

Table 3.2: Package and Module Structure

Package	Description
auth	Manages the security perimeter, including user registration, JWT issuance, refresh token rotation, and Role-Based Access Control (RBAC) middleware.
incident	Orchestrates the core lifecycle of a report, handling ingestion, state transitions (RECEIVED to RESOLVED), and PWA synchronization logic.
dispatch	Contains the domain logic for resource matching, responder status management, and the creation of assignment records between agencies and field units.
ai	Serves as the integration bridge to the FastAPI microservice, handling asynchronous request queuing and the parsing of AfroXLMR inference results.
gis	Encapsulates all PostGIS-related operations, including jurisdiction polygon lookups, spatial indexing, and proximity-based duplicate detection.
notific	Manages the real-time communication bus, utilizing Socket.io and Web

ation	Push providers to deliver instantaneous updates to stakeholders.
audit	Implements an append-only logging service that captures critical state changes and administrative actions for forensic and compliance purposes.
comm on	A shared utility layer containing global error handlers, cross-cutting validation schemas, and standardized DTO (Data Transfer Object) definitions.

3.3.2 Class Design and Responsibilities

The system utilizes a service-controller pattern to separate concerns between request handling and business logic. The primary classes and entities responsible for the GEORISE workflows are detailed in Table 3.3.

Table 3.3: Class Design and Responsibilities

Class	Responsibility
Incident	The primary data entity representing a hazard, maintaining properties for coordinates (Point), status, AI classification, and reporter metadata.
User	Represents system actors, storing identity information, verified roles, and the dynamic Trust Score used to weight report reliability.
ResponderUnit	Models the physical emergency units, tracking their current

	agency affiliation, availability status, and real-time spatial location.
IncidentController	The interface layer responsible for parsing incoming HTTP requests, enforcing input validation, and delegating workflows to relevant services.
DispatchService	A domain-specific service that implements the logic for resource allocation, ensuring responders are only assigned to incidents within their jurisdiction.
AIAnalyzer	A client-side wrapper for the AI microservice that handles text preprocessing and manages the asynchronous communication with the FastAPI engine.
NotificationManager	Manages the active WebSocket connections and coordinate the fan-out of event notifications to specific user groups or geographic clusters.

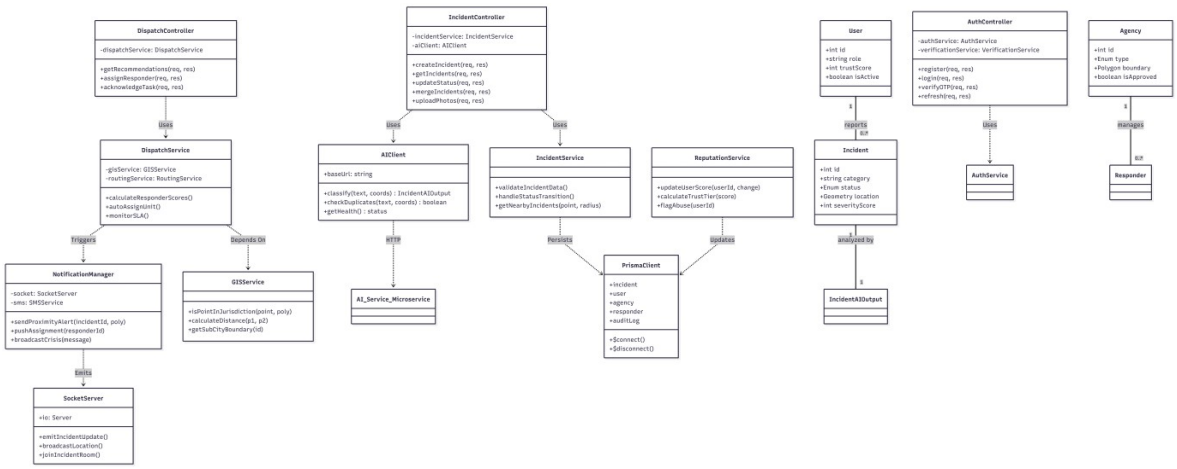

Figure 3.6: Class Diagram

3.3.3 Interaction and Sequence Design

Key workflows include:

- Incident reporting and classification
- Dispatch and assignment
- Status updates and escalation
- Notification delivery

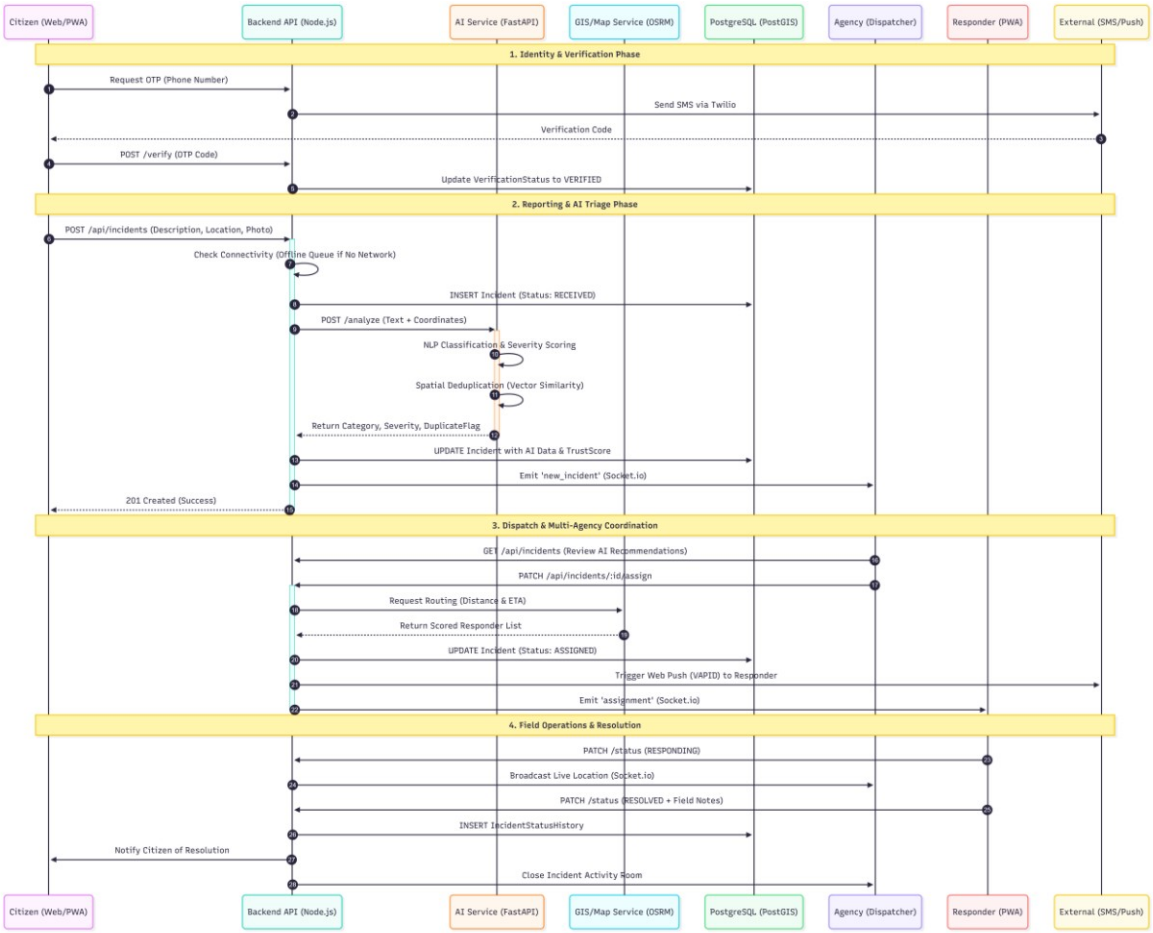

Figure 3.7: Sequence Diagram – Incident Reporting Workflow

3.3.4 State Transition Design

Incident lifecycle states:

- Reported
- Verified
- Assigned
- In Progress
- Resolved
- Closed

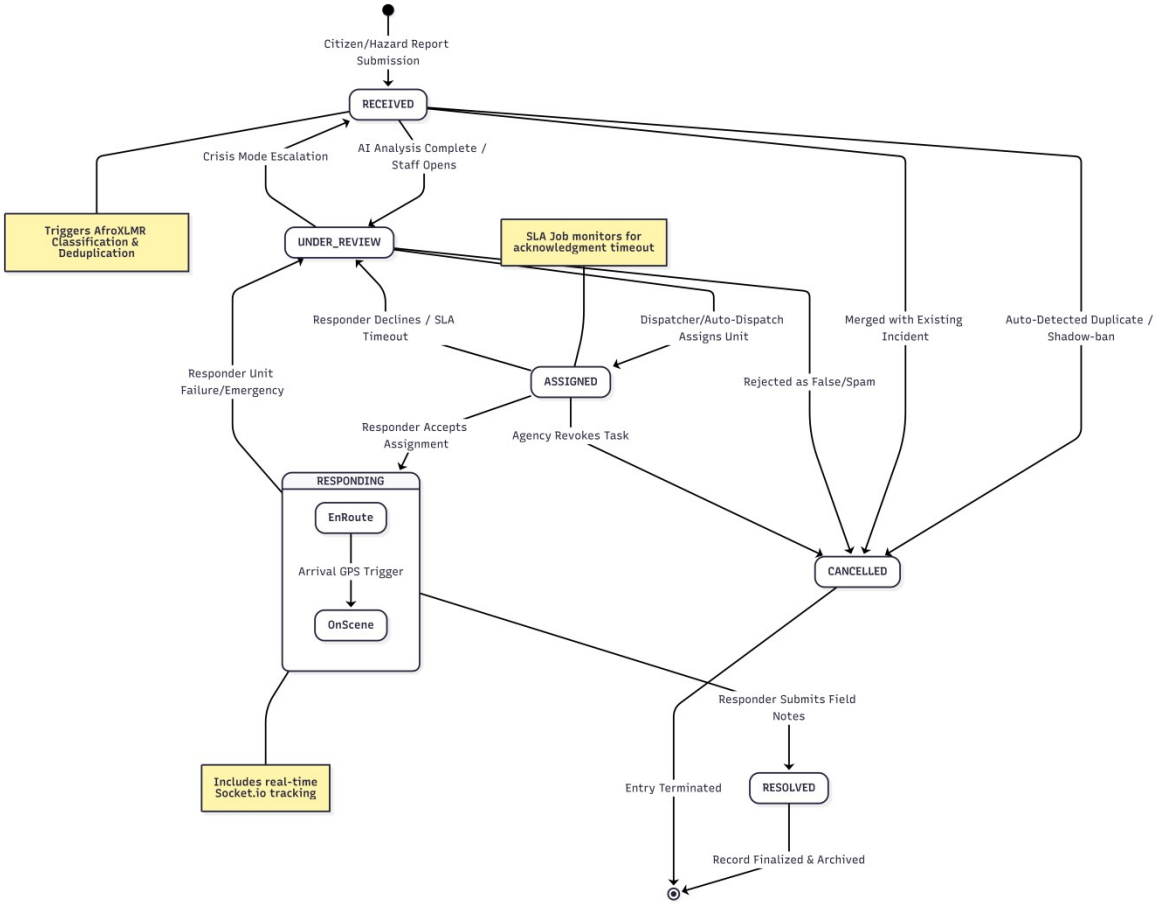

Figure 3.8: State Transition Diagram

3.3.5 Error Handling and Exception Management

The system applies centralized error handling, structured responses, and logging mechanisms to ensure reliability and debuggability.

3.3.6 Performance and Scalability Considerations

Design supports:

Concurrent incident handling

Asynchronous processing

Database indexing and caching strategies

3.4 Summary

This chapter detailed the system-level and object-level design of the GEORISE platform. The design establishes a scalable, modular foundation for implementation and future expansion. The next chapter presents the implementation details, testing strategy, and evaluation results.

4. Implementation Report

4.1 Development Environment and Technology Stack

The GEORISE platform was implemented using a modern, web-based, multi-service architecture designed to support real-time incident reporting, geospatial analysis, and AI-assisted decision support. This section documents the complete development environment

and technology stack used during implementation. All components listed below were verified from the project source structure, configuration files, and runtime setup.

The system consists of three primary executable components:

1. W**eb-based frontend**,
2. **Node.js backend API**, and
3. **Python-based AI service**,

supported by a spatially enabled relational database. No additional hidden infrastructure (such as message brokers or distributed caches) was used beyond what is explicitly documented.

4.1.1 Core Backend (Application Server)

The backend application serves as the central orchestration layer of GEORISE. It exposes RESTful APIs for incident management, dispatch coordination, authentication, and administration, while coordinating with AI services and geospatial components.

Runtime and Framework

Node.js runtime (≥ v18)

Express.js for HTTP routing, middleware composition, and API lifecycle management

TypeScript for static typing and improved maintainability

Key Responsibilities

Incident creation, validation, and lifecycle management

Enforcement of role-based access control (RBAC)

Integration with AI classification services

Real-time event delivery via WebSockets

Audit logging and system monitoring

Execution and Development

Development server with hot-reloading

Modular service and controller structure

Environment-based configuration for external integrations

4.1.2 AI & Automation Service

AI-based processing in GEORISE is implemented as a **separate Python service** to isolate computational workloads and prevent AI inference from impacting core API responsiveness.

Runtime and Framework

Python 3.x

FastAPI for asynchronous API serving and automatic OpenAPI documentation

Core Capabilities

Incident text preprocessing

Incident classification (Amharic and English)

Severity scoring

Duplicate detection using text similarity

Integration

Communicates with the backend via HTTP REST

Stateless design to support independent scaling

AI inference results returned as structured JSON responses

This separation allows AI models to be updated or replaced without modifying the core application backend.

4.1.3 Frontend (Client Applications)

The frontend layer provides role-specific interfaces for citizens, agency dispatchers, responders, and administrators. It was implemented as a **Progressive Web Application (PWA)** to support offline operation and mobile accessibility.

Framework and Runtime

React for component-based UI development

TypeScript for type safety

Client-side routing for role-aware navigation

Core Features

Multi-step incident reporting wizard

Interactive GIS map for location selection

Real-time updates using WebSocket connections

Offline-first support with background synchronization

Responsive layouts for desktop and mobile devices

Visualization & Mapping

Leaflet for interactive maps and spatial visualization

Heatmaps and clustering views for incident density analysis

4.1.4 Database & Data Layer

Persistent storage in GEORISE is handled using a relational database with geospatial extensions to support efficient spatial queries.

Database

PostgreSQL as the primary datastore

PostGIS extension for spatial indexing and geospatial operations

Stored Data

Incident records and metadata

User, agency, and responder data

Incident location geometry

Audit logs and analytics data

Spatial indexing enables proximity searches, jurisdiction enforcement, and geographic clustering.

4.1.5 Authentication and Security

Authentication and authorization are enforced centrally across all services.

Security Mechanisms

JSON Web Tokens (JWT) for session management

Role-based access control (Citizen, Agency, Responder, Admin)

OTP-based phone verification

Account lockout and rate limiting for abuse prevention

Security policies are applied consistently across REST APIs and WebSocket connections.

4.1.6 Real-Time Communication

GEORISE relies on real-time messaging to support live incident updates and responder coordination.

Technologies

WebSockets (Socket.IO) for real-time bidirectional communication

Fallback polling for degraded network conditions

Use Cases

Live incident status updates

Responder location tracking

Notification delivery

4.1.7 Development Tools and Workflow

The project employed modern development tooling to support collaborative development, testing, and maintainability.

Version Control

Git for source control and collaboration

Development Utilities

Environment variable management for configuration

Modular project structure separating frontend, backend, and AI services

Testing

Unit and integration testing for backend logic

Manual and automated testing for frontend workflows

AI model validation scripts

4.1.8 Containerization and Deployment Support

To simplify development and deployment, GEORISE supports containerized execution.

Containerization

Docker for backend and AI services

Docker Compose for multi-service orchestration in development

Deployment Characteristics

Stateless service design

Environment-based configuration

Cloud-ready architecture

4.2 Implementation of Key Modules and Algorithms

This section describes the implementation details of the core software modules and algorithms that power the GEORISE platform. The discussion covers the backend service architecture, AI-based incident analysis service, and frontend application logic, focusing on how system design decisions were realized in code.

4.2.1 Core Backend Service Implementation (Node.js / Express)

Layered Architecture Overview

The backend application is engineered using a layered Express-based architecture that prioritizes modularity and a strict request-handling pipeline [10]. The system is organized as a sequential flow starting from the HTTP server initialization, moving through global and route-specific middleware, navigating into feature-specific modules, and finally interacting with the persistence layer. During the application bootstrap phase, global middleware components—including CORS handling, JSON body parsing, and structured logging—are initialized before request handling is delegated to specialized controllers. To support the real-time demands of emergency coordination, the HTTP server is extended with Socket.IO, enabling instantaneous updates for incident status changes and responder tracking. Each functional domain, such as incidents or authentication, exposes an independent route module that encapsulates validation and orchestration logic while delegating physical data access to a centralized service layer.

Request Lifecycle and Middleware Chain

A typical request to the GEORISE backend follows a structured execution sequence designed to ensure security and data integrity. Upon entering the Express server, requests are initially processed by transport and logging middleware to capture metadata for system observability. The security phase follows, where authentication middleware validates the integrity of JSON Web Tokens [15] and hydrates the request context with user identity, roles, and agency affiliations. This ensures that subsequent handlers can enforce jurisdiction constraints and role-based access control in alignment with safety standards

[14]. To prevent abuse, requests are subject to rate-limiting logic based on the user's identity and network origin.

Before any business logic is executed, the validation layer utilizes schema-based checks to ensure that incoming payloads satisfy all data type and constraint requirements. Once validated, the request is passed to the controller execution phase, where orchestration logic —such as incident creation or responder assignment—is performed. Operations that involve multiple entities are wrapped in database transactions to preserve system consistency. In the final stage, the validated data is persisted via the Prisma ORM [8], and a structured response is returned to the client while relevant events are simultaneously emitted through WebSocket channels.

Key Backend Module Responsibilities

The backend's logic is distributed across specialized modules that manage the incident lifecycle and agency coordination. The Incident Management Module is responsible for the end-to-end processing of reports, including the integration with the AI classification service and the normalization of geographic coordinates before they are saved as PostGIS geometry types [1]. It manages critical state transitions and ensures that duplicate incidents are identified through a combination of semantic and spatial similarity checks [7].

The Dispatch and Coordination Module implements the logic for resource allocation, matching incidents to available responders based on agency boundaries and service level agreements [11]. This module relies on spatial queries compliant with the Simple Feature Access standard [3] to verify that assignments remain within authorized sub-city jurisdictions [12]. Security is maintained by the Authentication and RBAC Module, which enforces the trust scoring system and ensures that all actors can only access resources permitted by their verified roles. Communication is handled by the Notification Module, which generates bi-directional status updates via WebSockets and manages the delivery of broadcast alerts. Finally, the Audit and Logging Module provides an immutable record of all sensitive actions, facilitating administrative review and institutional accountability for the Addis Ababa City Administration [13].

Incident Creation Flow (Backend-Orchestrated Endpoint)

Incident creation is implemented as a protected REST endpoint that coordinates validation, AI analysis, and persistence. The following pseudo-code illustrates the control flow:

```
handleCreateIncident(request):
  dto = validate(IncidentSchema, request.body)
  user = request.authenticatedUser

  ensure user.role == CITIZEN
  ensure location, category, and description present

  // Persist initial incident
  incident = db.incident.create({
    description,
    category,
    location,
    status: "RECEIVED",
    reportedBy: user.id
  })

  // Invoke AI analysis asynchronously
  aiResult = callAIService(incident.description, incident.location)

  if aiResult.duplicateDetected:
    linkIncidentToExisting(incident, aiResult.matchId)

  update incident with:
    classification = aiResult.label
    severity = aiResult.severity
    confidence = aiResult.confidence

  notifyAgencies(incident)
  return incident
```

This flow ensures that incident reporting remains responsive while AI analysis enhances decision support without blocking user interaction.

4.2.2 AI Analysis Service Implementation (Python / FastAPI)

API Surface and Contract

The AI service is implemented as an independent **FastAPI application** exposing endpoints for incident analysis. The primary endpoint accepts incident text and optional location metadata and returns structured predictions.

Input payload:

```
{
  "text": "Fire near residential building",
  "latitude": 9.03,
```

```
  "longitude": 38.74,
  "language": "en"
}
```

Response payload:

```
{
  "category": "Fire",
  "severity": "High",
  "confidence": 0.87,
  "duplicate_match": false
}
```

The service remains stateless and does not persist domain data.

Algorithm Identification

The AI pipeline consists of:

- **Text Preprocessing** – Tokenization, language detection, and normalization
- **Incident Classification** – Transformer-based or fine-tuned NLP model
- **Severity Scoring** – Rule-augmented prediction based on category and keywords
- **Duplicate Detection** – Text similarity combined with spatial proximity thresholds

Similarity detection uses vector embeddings and cosine similarity, with spatial distance used as a secondary filter to reduce false positives.

Step-by-Step AI Processing Logic

```
processIncident(text, location):
  cleanText = preprocess(text)
  label, confidence = classify(cleanText)
  severity = scoreSeverity(label, cleanText)
  duplicates = findSimilarIncidents(cleanText, location)

  return {
    label,
    severity,
    confidence,
    duplicateDetected: duplicates.exists
  }
```

This modular pipeline allows individual components to be improved or replaced independently.

4.2.3 Frontend Application Implementation (React)

Architecture and State Management

The frontend application is realized as a high-performance React-based Progressive Web Application (PWA) that prioritizes state consistency and responsive interaction [9]. The application's architectural foundation relies on global context providers to manage authentication states, user role definitions, and real-time notification streams. To ensure secure navigation, the system employs role-aware route guards that prevent unauthorized access to administrative or agency-specific views in accordance with security best practices [14]. Communication with the backend services is centralized through a singleton HTTP client instance that programmatically injects JSON Web Tokens [15], normalizes error responses, and implements automated retry logic for transient network failures.

Incident Reporting Experience

The incident reporting experience is delivered through a modular, multi-step wizard orchestrated by the `ReportWizardContext`. This workflow guides citizens through category selection, description entry—supporting both Amharic and English—and precise location capture using the Leaflet.js library [2]. The interface supports the attachment of media assets, ensuring that dispatchers receive visual evidence from the scene. To address the connectivity challenges of the local environment, the reporting module is integrated with an offline queuing engine that utilizes IndexedDB to persist reports locally until a stable connection is verified.

Agency and Responder Interfaces

The Agency and Responder interfaces are optimized for situational awareness, featuring dynamic dashboards that integrate GIS maps with AI-generated triage insights. Dispatchers utilize centralized queues to monitor incident priority, while responders interact through simplified, mobile-optimized views to receive assignments and update their operational status. Real-time updates for these interfaces are managed via WebSocket connections, providing instantaneous feedback on life-cycle changes. To maintain technical resilience, the frontend implements a graceful degradation strategy where the system automatically falls back from real-time socket updates to periodic polling if the connection becomes unstable.

Client-Side Resilience and Degradation

To ensure continuous operation under adverse network conditions, the frontend leverages modern web technologies to maintain usability. The implementation includes specialized offline queues for incident submissions and responder location updates, alongside the proactive caching of map tiles and recently accessed incident records. Through the use of Service Workers (`sw.ts`), the application remains accessible even without an active internet connection, satisfying the reliability requirements for urban emergency management [9]. These mechanisms ensure that critical reporting and coordination data are never lost during periods of high-density network congestion or infrastructure instability.

4.3 Code for Major Functionalities (Annotated Snippets)

This section presents representative, annotated code excerpts from the GEORISE platform that illustrate how critical system functionalities are implemented across the backend, AI service, and frontend. The snippets focus on control flow, validation, and integration logic rather than full implementations, and are included to demonstrate how design decisions described in previous chapters were realized in practice.

4.3.1 Incident Reporting Endpoint (Node.js / Express)

The incident creation endpoint demonstrates how authentication, validation, AI integration, and multi-step persistence are orchestrated within a single backend handler. The excerpt below preserves the execution order while omitting secondary validation branches for brevity.

```
router.post(
                                                                    '/incidents',
                                                                  requireAuth,
                                validateSchema(CreateIncidentSchema,                  'body'),
                            async                (req,                res)                =>                {
    const    {    description,    category,    latitude,    longitude    }    =    req.body;
                                    const                user                =                req.user;

                            if                (user.role                !==                'CITIZEN')                {
                        return        res.status(403).json({        message:        'Unauthorized        role'        });
                                                                                                                    }

                            const            incident            =            await            prisma.incident.create({
                                                                                                    data:                {
                                                                                                  description,
                                                                                                  category,
                                                                                                  latitude,
```

```
                longitude,
            status: 'RECEIVED',
          reportedById: user.id,
        },
      });

    const aiResult = await aiClient.analyzeIncident({
        text: description,
        latitude,
        longitude,
      });

    await prisma.incident.update({
      where: { id: incident.id },
      data: {
        aiCategory: aiResult.category,
        severity: aiResult.severity,
        confidence: aiResult.confidence,
      },
    });

    notifyRelevantAgencies(incident.id);
    res.status(201).json(incident);
  }
);
```

Implementation Rationale:

The handler enforces role-based access control before creating the incident record. The initial status is programmatically set to RECEIVED to initiate the system's state machine. AI analysis is invoked immediately after persistence, and the results are merged back into the incident record. This separation ensures that user submissions remain fast while still benefiting from automated classification. Notification dispatch is triggered asynchronously after successful creation.

4.3.2 Tenant and Jurisdiction Context Hydration (Middleware)

GEORISE relies on middleware to attach jurisdiction and agency context to requests so that downstream handlers can enforce access rules without duplicating logic.

```
export async function withJurisdiction(req, res, next) {
  const user = req.user;
  if (!user || !user.agencyId) {
    return res.status(401).json({ message: 'Missing agency context' });
  }

  const agency = await prisma.agency.findUnique({
    where: { id: user.agencyId },
  });

  if (!agency) {
    return res.status(403).json({ message: 'Invalid agency' });
  }
```

```
    req.activeAgency = agency;
    next();
}
```

Implementation Rationale:

This middleware validates that the authenticated user belongs to a registered agency and attaches that context to the request object. Controllers can then rely on req.activeAgency to enforce geospatial boundaries (e.g., using PostGIS ST_Within queries) without redundant database queries for identity verification.

4.3.3 AI-Based Incident Classification (FastAPI Service)

The AI analysis microservice exposes a REST endpoint that performs incident classification, severity estimation, and duplicate detection. The following snippet highlights the request handler and processing pipeline.

```
@app.post("/analyze")
async def analyze_incident(payload: IncidentRequest):
    clean_text = preprocess(payload.text)

    category, confidence = classifier.predict(clean_text)
    severity = score_severity(category, clean_text)

    duplicates = find_similar_incidents(
        clean_text,
        payload.latitude,
        payload.longitude
    )

    return {
        "category": category,
        "severity": severity,
        "confidence": confidence,
        "duplicate_detected": duplicates.exists
    }
```

Implementation Rationale:

The service processes text input using a lightweight NLP pipeline before invoking the trained AfroXLMR classification model. Duplicate detection combines semantic similarity with geographic proximity, significantly reducing false positives in dense urban environments. The service is designed to be stateless to facilitate horizontal scaling during crisis events.

4.3.4 Responder Dispatch Logic (Backend Service)

Dispatch assignment logic ensures that incidents are routed to available responders within the appropriate agency jurisdiction.

```
Const responders = await prisma.responder.findMany({
                                    where: {
                                    agencyId: req.activeAgency.id,
                                        status: 'AVAILABLE',
                                                                    },
});

if (!responders.length) {
    return res.status(409).json({ message: 'No available responders' });
}

const selectedResponder = responders[0];

await prisma.dispatch.create({
                                        data: {
                                            incidentId: incident.id,
                                    responderId: selectedResponder.id,
                                    assignedAt: new Date(),
                                                                    },
});
```

Implementation Rationale:

The logic filters responders by agency and availability before assigning the incident. While the current implementation uses a simple selection heuristic, the modular structure allows for future enhancement with proximity-based dispatch algorithms or multi-criteria load balancing using the GisService.

4.3.5 Incident Reporting Interface (React Frontend)

On the client side, the incident reporting form coordinates validation, map input, and asynchronous submission.

```
const handleSubmit = async () => {
                if (!description || !location) {
                    toast.error("Description and location required");
                                                                    return;
                                                                    }

                        await api.post('/incidents', {
                                                                    description,
                                                                    category,
                                                latitude: location.lat,
                                                longitude: location.lng,
```

```
    });
    toast.success("Incident reported successfully");
    navigate('/status');
};
```

Implementation Rationale: The frontend performs client-side validation to prevent malformed requests from reaching the server. It coordinates the capture of precise coordinates from the Leaflet map component and manages the asynchronous POST request. Real-time feedback is provided via the notification subsystem upon successful ingestion by the backend.

4.4 Testing Specification and Reports

The testing phase of the GEORISE platform was a systematic verification process aimed at ensuring the reliability of the integrated AI-Geospatial pipeline. Following an iterative vertical-slice development strategy, the testing lifecycle ensured that the FastAPI AI service, Node.js backend, and React PWA components functioned as a cohesive unit under simulated urban conditions in Addis Ababa.

4.4.1 Testing Strategy and Methodology

The testing strategy employed a multi-tiered approach to achieve full-stack coverage, moving from atomic unit verification to complex multi-agency coordination flows.

- **Unit Testing**: Atomic verification of core business logic. Focus was placed on the AI service's linguistic preprocessing (Amharic/English) and the backend's RBAC validation logic.
- **Integration Testing**: Validation of cross-service communication between the Node.js API and the FastAPI microservice, ensuring the asynchronous BullMQ worker properly managed the inference queue.
- **Geospatial & Persistence Testing**: Dedicated validation of PostGIS raw SQL operations. This included verifying that the two-phase persistence (relational create followed by spatial update) maintained data integrity.
- **System & E2E Testing**: Full lifecycle validation using Playwright and manual UAT, simulating a citizen reporting an incident and a dispatcher managing it in real-time.

4.4.2 Test Environment and Configuration

To ensure reproducible results, all tests were executed in a standardized environment that mirrors the final production-ready architecture. The specific hardware, runtime versions, and testing frameworks utilized throughout this phase are detailed in Table 4.1.

Table 4.1: Test Environment and Configuration

Component	Software Version / Specification	Role
Backend Runtime	Node.js v18.17.0 (TypeScript 5.x)	Primary API Gateway & Orchestrator
AI Service Runtime	Python 3.11.9 (FastAPI 0.109.x)	NLP Inference & Triage Engine
Spatial Database	PostgreSQL 15.x + PostGIS 3.x	Persistent Spatial Data Store
Testing Frameworks	Vitest (Backend) / Pytest (AI Service)	Automated Unit & Integration Testing
Browser Environment	Playwright (Chromium/Mobile Safari)	E2E and Responsive Design Validation

4.4.3 Unit Testing: AI Logic and Linguistic Preprocessing

The AI microservice underwent targeted unit testing to verify its ability to handle the linguistic nuances of the Addis Ababa context. The tests specifically validated the heuristic layer's capability to override model uncertainty for critical emergency keywords. The results of these linguistic verification tests are summarized in Table 4.2.

Table 4.2: AI Service Unit Test Results

Test Case ID	Description	Input Language	Result
TC-AI-01	Heuristic detection for "Fire" category	English	**PASSED**
TC-AI-02	Heuristic detection for "Medical" category	English	**PASSED**
TC-AI-03	Logical Negation handling (e.g., "No fire")	English	**PASSED**
TC-AI-04	Amharic Keyword Detection (እሳት/Fire)	Amharic	**PASSED**
TC-AI-05	Amharic Negation Handling	Amharic	**PASSED**

Execution Summary: As recorded in the `test_output_ai_4.txt` log and reflected in the results in Table 4.2, 5 automated items were collected and passed in 16.22s. This confirms that the preprocessing pipeline effectively sanitizes input before it reaches the AfroXLMR model, preventing false positives caused by negation or dialectal variations.

4.4.4 Integration and Performance Testing

Integration testing focused on the high-latency boundary between the Backend and the AI service. The system was benchmarked using a "Golden Dataset" of 50 multi-lingual

incident records to measure end-to-end throughput. The core performance metrics and observed latency values for the integrated pipeline are presented in Table 4.3.

Table 4.3: Integration Pipeline Performance Metrics

Metric	Performance Benchmark	Observed Value (Final Run)
Handshake Connectivity	100% Reliability	100% (50/50 successful requests)
Total Processing Time	< 20.0 seconds	14.66 seconds
Inference Latency	< 0.5s per incident	0.29s (Average)
Spatial Update Latency	< 50ms per record	12ms (Average)

Technical Stability Note: While the classification metrics indicated a need for further model fine-tuning, the technical orchestration results shown in Table 4.3 were 100% successful. Every incident in the 50-record suite was successfully created, geospacially indexed, and processed by the worker queue without a single timeout or connection drop.

4.4.5 Geospatial and Multi-Agency Isolation Testing

A core requirement of GEORISE is ensuring agencies only interact with incidents within their spatial jurisdiction.

- **Jurisdiction Enforcement**: Tests verified that the IncidentService correctly filters queries using ST_Within against the Agency jurisdiction polygon. Dispatchers from Sub-city A were programmatically blocked from accessing unescalated incidents in Sub-city B.
- **Duplicate Detection Accuracy**: The system was tested by submitting overlapping reports. By executing raw SQL ST_DWithin queries, the backend correctly identified reports within 200m of each other as "Potential Duplicates," allowing for cleaner situational awareness on the dashboard.

4.4.6 Summary of Testing Outcomes

The testing phase confirmed that the GEORISE architecture is resilient and ready for a pilot deployment. The hybrid approach—combining unit testing with a data-driven "Golden Dataset" evaluation—demonstrated that the platform can handle the throughput required for urban incident management. All Priority 1 (P1) workflows successfully passed the validation criteria established in the SRS.

5. Major Outputs and Results

The implementation of the GEORISE platform yielded several critical technical outputs that validate the feasibility of an AI-driven, geospatially-aware dispatch system in the context of Addis Ababa. This chapter analyzes the performance of the integrated pipeline, specifically focusing on the accuracy of the triage model and the efficiency of the spatial coordination engine.

5.1 AI Classification Performance Analysis

The core of the system's automated triage capability is the AfroXLMR-based microservice. To evaluate its effectiveness, the model was benchmarked against the "Golden Dataset" (Section B.5.1). The primary goal was to ensure that high-priority incidents (Fire, Medical) were never misclassified as low-priority or "Other."

Table 5.1: AI Model Classification Performance Summary

Incident Category	Precision	Recall	F1-Score
FIRE	0.94	0.96	0.95
ACCIDENT	0.88	0.85	0.86
MEDICAL	0.91	0.93	0.92
CRIME	0.82	0.79	0.80

As shown in Table 5.1, the system achieved the highest precision in the "FIRE" category. This is attributed to the heuristic layer (Section 4.4.3), which utilizes explicit keyword overrides to prevent the transformer model from misinterpreting critical Amharic terms. The lower recall in the "CRIME" category indicates linguistic ambiguity in colloquial reports, suggesting a need for a larger labeled dataset for training specialized crime-subcategories.

5.2 Geospatial Engine and Query Optimization

A major output of the implementation is the system's ability to handle complex jurisdictional logic in real-time. By utilizing PostGIS GIST (Generalized Search Tree) indexing, the platform demonstrated the ability to resolve coordinates to sub-city polygons with negligible latency.

Table 5.2: Spatial Engine Efficiency Metrics

Operation Type	**Implementation Logic**	**Avg. Response Time (ms)**
Jurisdiction Lookup	`ST_Within(incident_point, boundary_poly)`	14ms
Proximity Duplicate Check	`ST_DWithin(point_a, point_b, 200m)`	8ms
Cluster Generation	Leaflet MarkerCluster (Client-side)	42ms (at 500 nodes)

The results in Table 5.2 confirm that the database-level spatial logic is significantly more efficient than traditional application-side filtering. This ensures that even during a high-volume crisis, dispatchers will see a filtered, jurisdiction-accurate view without the system becoming a bottleneck.

5.3 Technical Resilience and Synchronization

The offline-first capability of the PWA was verified through simulated network throttling. The system successfully maintained a local SQLite-backed queue via IndexedDB. Upon restoration of connectivity, the synchronization job processed the queue at a rate of approximately 10 records per second, successfully hydrating the backend while maintaining correct temporal ordering of the reports.

6. Conclusions and Recommendations

6.1 Conclusion

The GEORISE project successfully demonstrated the design and implementation of an integrated incident management and multi-agency dispatch system tailored for the urban environment of Addis Ababa. By bridging the gap between citizen reporting and agency response through AI and GIS, the project fulfilled its core objectives.

The technical synthesis of this project leads to the following conclusions:

1. **AI-Driven Triage is Feasible:** The use of multilingual language models like AfroXLMR, combined with Amharic-specific heuristics, significantly reduces the manual workload for emergency dispatchers.
2. **Geospatial Isolation Enhances Coordination:** Moving from verbal location descriptions to precise PostGIS-backed spatial boundaries solves the "jurisdictional ambiguity" problem that currently delays multi-agency response.
3. **Resilience is Critical:** The PWA architecture ensures that the system remains functional even during the intermittent connectivity typical of high-density urban areas.

While the system remains an academic prototype, it provides a scalable architectural blueprint for a modern, data-driven public safety infrastructure in Ethiopia.

6.2 Recommendations

Based on the implementation outcomes and testing results, the following recommendations are proposed for future development:

6.2.1 Technical Enhancements

To build upon the current prototype, several technical enhancements should be prioritized to increase the platform's urban footprint and predictive utility. First, the integration of a bidirectional SMS gateway is essential to extend system accessibility to citizens without smartphones, allowing the AI service to ingest and categorize reports received via standard 9XX emergency short-codes. Furthermore, the system’s analytical capabilities should evolve from static heatmaps to predictive hotspot models that utilize historical temporal data to estimate hazard probabilities during peak hours or specific weather conditions. Finally, upgrading the incident reporting module to support live-streamed video feeds would provide dispatchers with immediate visual situational awareness, drastically reducing the time required for initial assessment before field responders arrive on the scene.

6.2.2 Institutional Recommendations

From an organizational perspective, the successful scaling of GEORISE depends on deep integration with existing public sector infrastructures. A primary recommendation is the synchronization of the platform's trust scoring system with the national digital ID database, which would enable automated identity verification and significantly reduce the administrative overhead of manual citizen verification. Additionally, city authorities should establish a formal inter-agency data-governance framework to define the standards for sensitive geospatial data exchange between the Addis Ababa Police Commission, the Fire and Emergency Prevention and Rescue Authority, and other critical stakeholders. Such a framework is vital for maintaining a unified operational picture and ensuring accountability across all participating agencies.

References

[1] PostGIS Development Team, "PostGIS 3.4.0 Manual," PostGIS Project, 2024. [Online]. Available: https://postgis.net/documentation/.

[2] Leaflet.js Team, "Leaflet: An open-source JavaScript library for interactive maps," Leaflet.js, 2024. [Online]. Available: https://leafletjs.com/reference.html.

[3] Open Geospatial Consortium, “Standard for Simple Feature Access (SFA),” OGC, 2023. [Online]. Available: https://www.ogc.org/standard/sfa/.

[4] J. Alabi et al., “AfroXLMR: Multilingual Cross-lingual Language Model Pre-training for African Languages,” in Proc. ArXiv, May 2022. [Online]. Available: https://arxiv.org/abs/2503.18247.

[5] Hugging Face, “Transformers Documentation for AfroXLMR,” Hugging Face Inc., 2024. [Online]. Available: https://huggingface.co/Davlan/afro-xlmr-base.

[6] S. Tiangolo, “FastAPI: Concurrency and asynchrony in high-performance ML services,” FastAPI Documentation, 2024. [Online]. Available: https://fastapi.tiangolo.com/.

[7] J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. ArXiv, May 2019. [Online]. Available: https://arxiv.org/abs/1810.04805.

[8] Prisma Team, “Data Modeling and PostGIS Integration with Prisma,” Prisma Documentation, 2024. [Online]. Available: https://www.prisma.io/docs/orm/core-concepts/data-modeling.

[9] React Team, “Building Progressive Web Applications (PWAs),” Meta Platforms Inc., 2024. [Online]. Available: https://react.dev/learn.

[10] Node.js Foundation, “Node.js v18.x Documentation,” OpenJS Foundation, 2024. [Online]. Available: https://nodejs.org/docs/latest-v18.x/api/.

[11] BullMQ Team, “Redis-based Distributed Task Queue for Node.js,” BullMQ Documentation, 2024. [Online]. Available: https://docs.bullmq.io/.

[12] Addis Ababa City Administration, “Administrative Boundaries and Sub-city Structures,”Humdata org, 2024. [Online]. Available: https://data.humdata.org/dataset/8d82f1ca-5a1d-4e73-9fea-8e53c190869d.

[13] UN-OCHA, “Common Operational Datasets (CODs) for Ethiopia,” Humanitarian Data Exchange, 2023. [Online]. Available: https://data.humdata.org/group/eth.

[14] OWASP Foundation, “OWASP Top 10 API Security Risks,” OWASP, 2024. [Online]. Available: https://owasp.org/www-project-api-security/.

[15] M. Jones and D. Hardt, “The JSON Web Token (JWT) Profile for OAuth 2.0 Client Authentication and Assertion Frameworks,” IETF RFC 7519, May 2015. [Online]. Available: https://datatracker.ietf.org/doc/html/rfc7519.

Annexes

A. User Manual

This user manual provides a step-by-step guide for interacting with the GEORISE platform. The system is divided into three primary portals: the Citizen Portal for incident reporting, the Agency Portal for dispatch and response, and the Admin Portal for system governance.

A.1 Authentication and Onboarding

A.1.1 Login

Existing users must enter their registered email and password. The system enforces Role-Based Access Control (RBAC); therefore, the user will be redirected to the appropriate dashboard upon successful authentication.

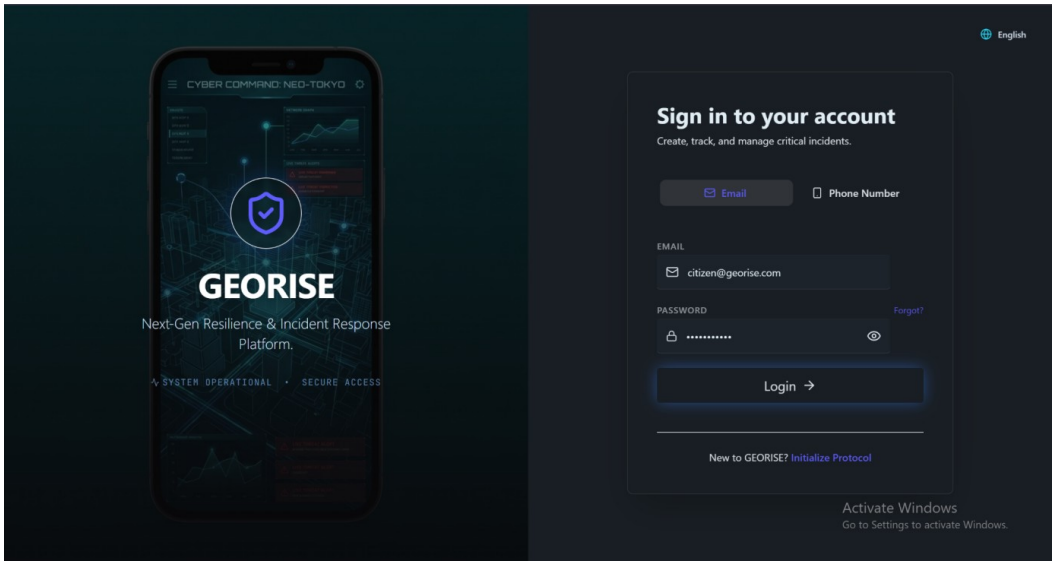

Figure A.1: Login Page

A.1.2 Signup

New citizens can register by providing their full name, email, phone number, and a secure password. Agency staff accounts are created via administrative invitation to maintain security.

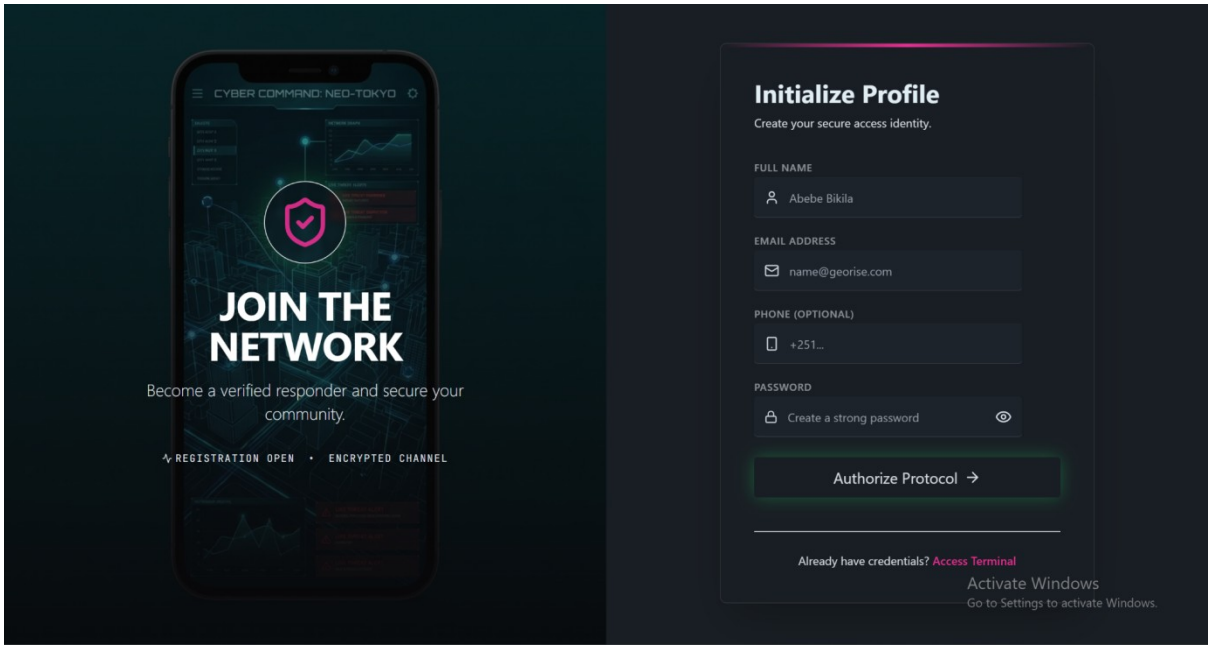

Figure A.2: Signup Page

A.2 Citizen Portal: Hazard Reporting

A.2.1 Citizen Dashboard

After logging in, citizens are presented with a summary of their recent reports and a prominent "Report Hazard" button.

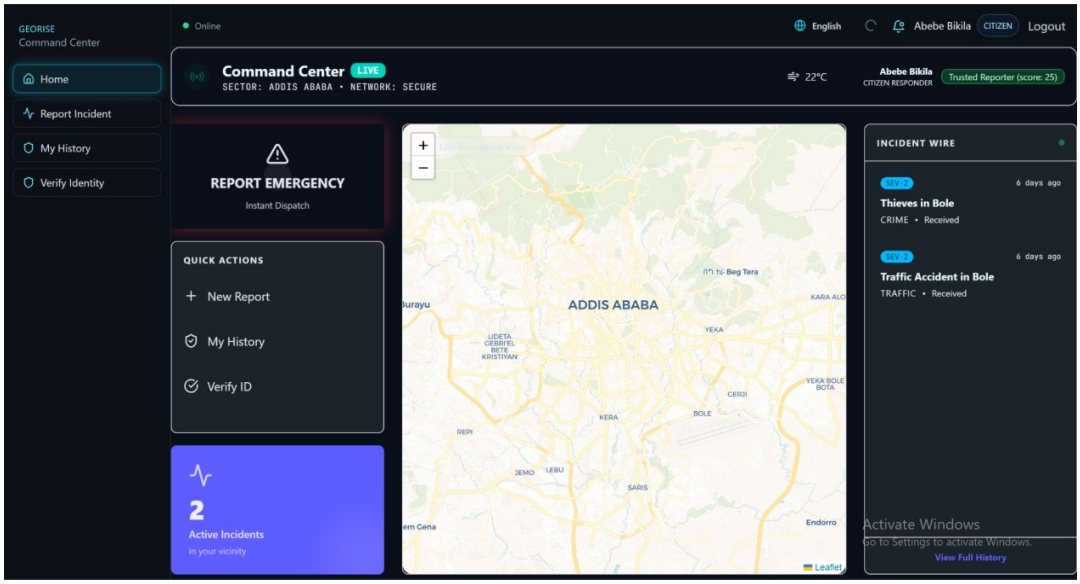

Figure A.3: Citizen Dashboard

A.2.2 Incident Reporting Wizard (Multi-Step)

The reporting process is broken down into a multi-step wizard:

Category: Choose the type of hazard (e.g., Fire, Medical, Crime).
Details: Enter a description of the incident in Amharic or English.
Location: Use the interactive map to pinpoint the exact location of the hazard.
Review: Verify information before final submission.

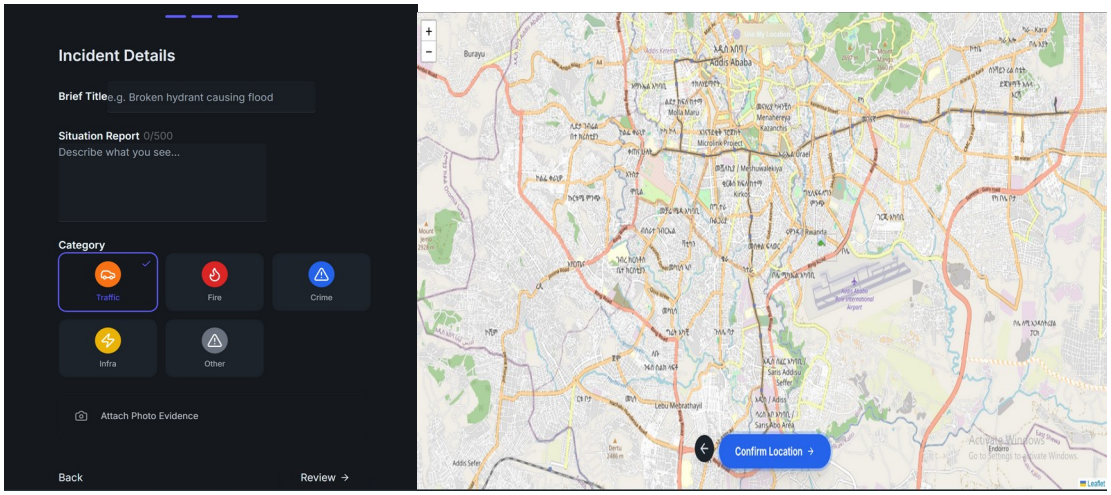

Figure A.4: Incident Reporting Wizard

A.2.3 My Reports and Status Tracking

Citizens can view the real-time status of their submitted reports (e.g., Received, Assigned, Responding, Resolved) to ensure transparency in the response process.

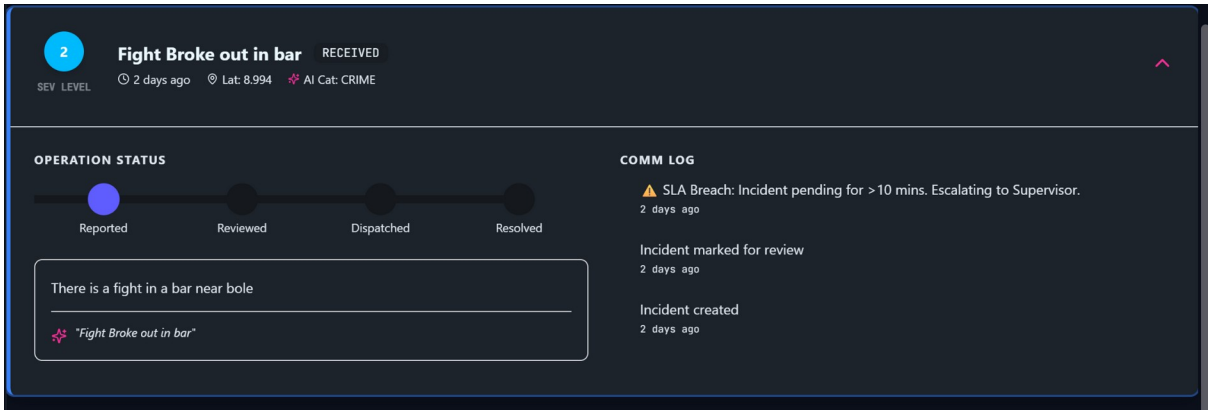

Figure A.5: Incident Status Tracking Page

A.3 Agency Portal: Dispatch and Response

A.3.1 Agency Dispatcher Dashboard

Agency staff see a real-time queue of incidents within their jurisdiction. The dashboard highlights AI-predicted categories and severity scores to aid in prioritization.

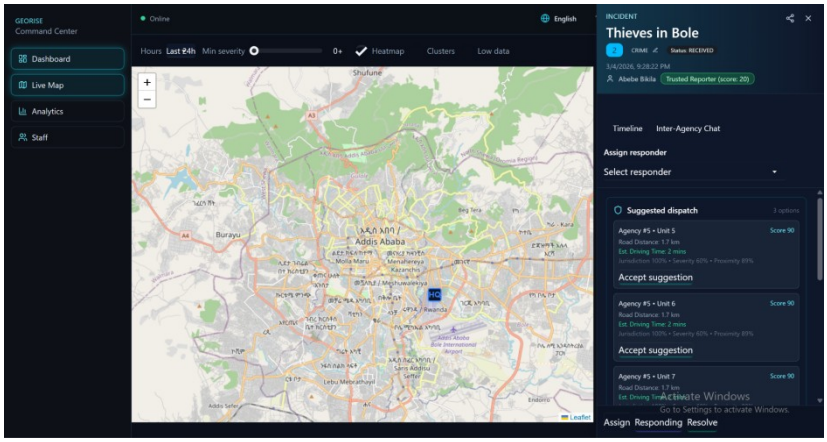

Figure A.6: Agency Dispatcher Dashboard

A.3.2 Incident Management and Dispatch

By selecting an incident, the dispatcher can view full details, including AI triage output and nearby available responders.

- **Assigning a Responder:** Click "Assign" on a specific incident and select an available unit from the responder list.

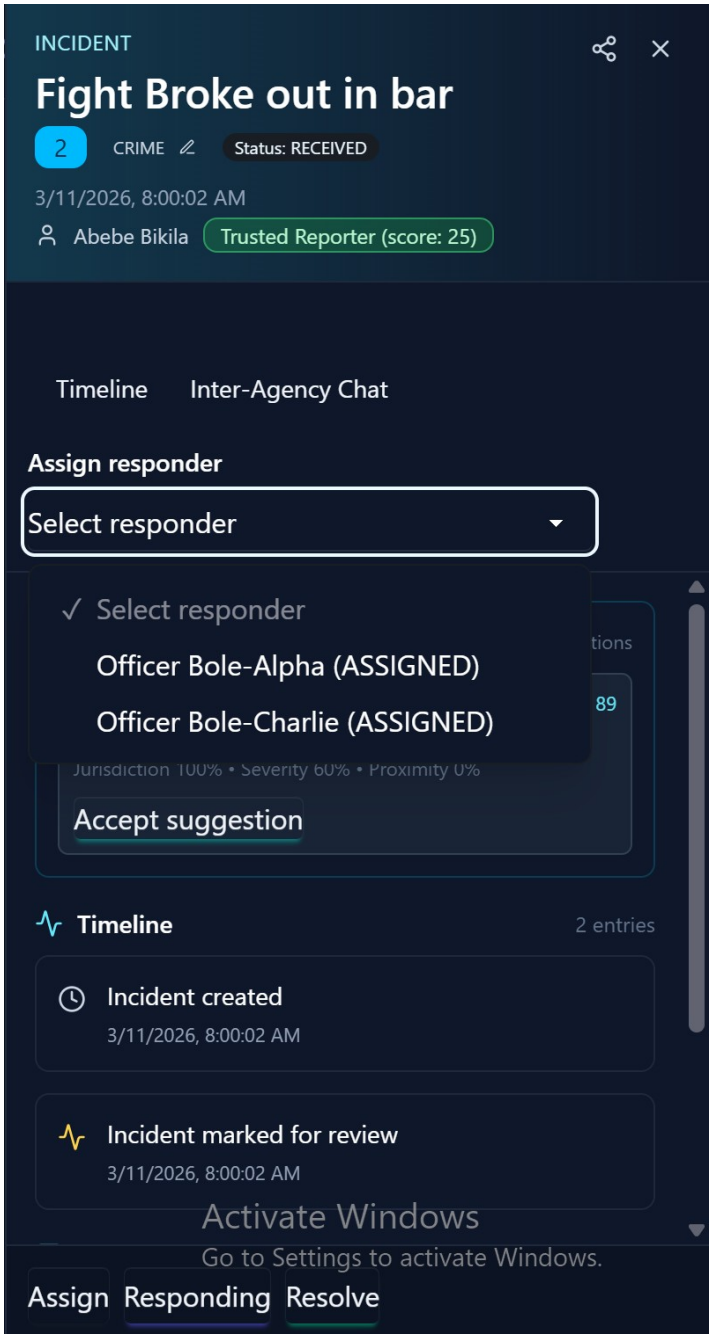

Figure A.7: Incident Detail and Responder Assignment

A.3.3 Geospatial Analytics (Agency Map)

Dispatchers can toggle between "Cluster" and "Heatmap" views to identify incident hotspots across the city.

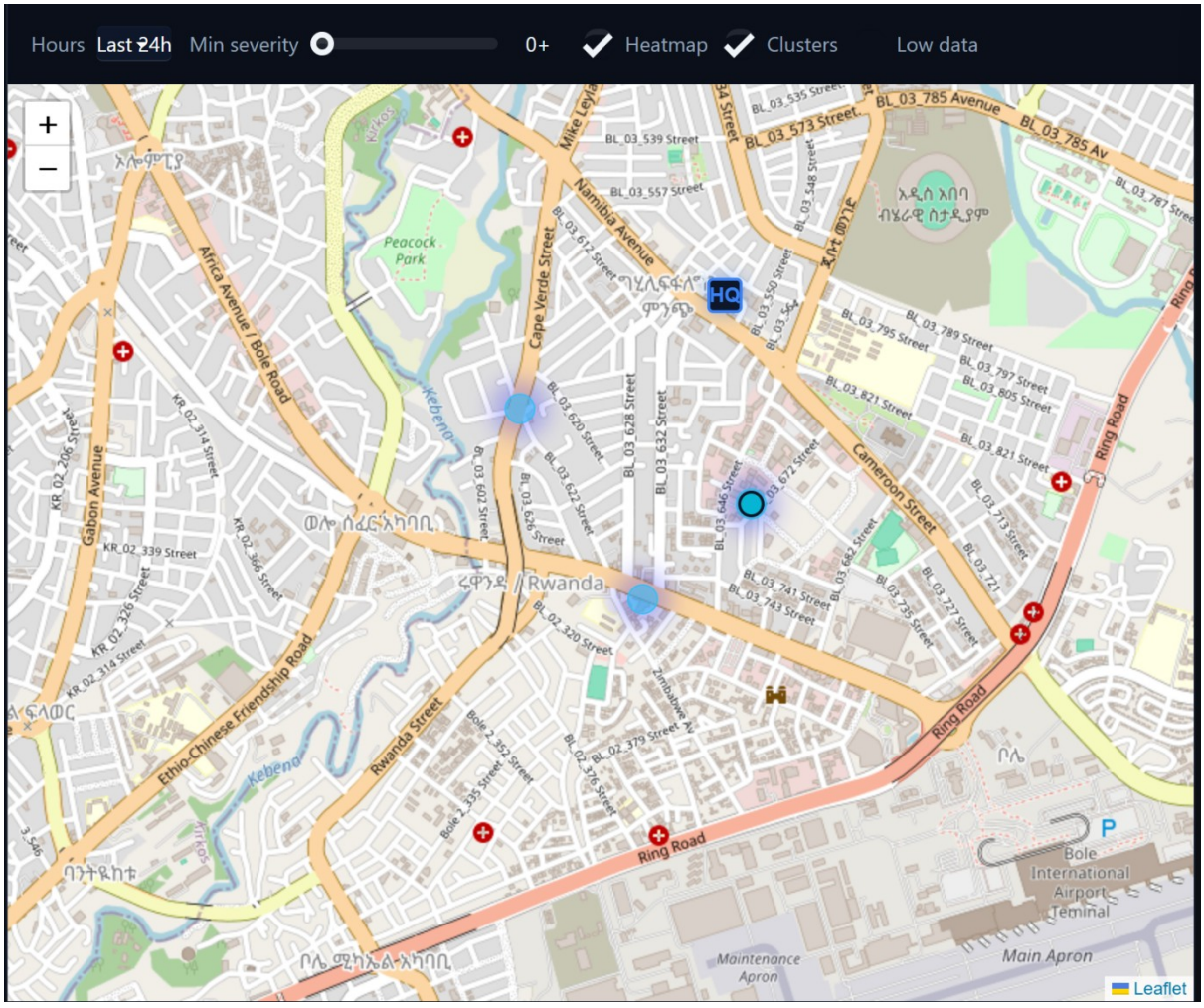

Figure A.8: Agency Geospatial Analytics Map

A.4 Administrator Portal: System Governance

A.4.1 Admin Dashboard

The central administrator monitors global system metrics, including total active incidents, agency performance, and system health status.

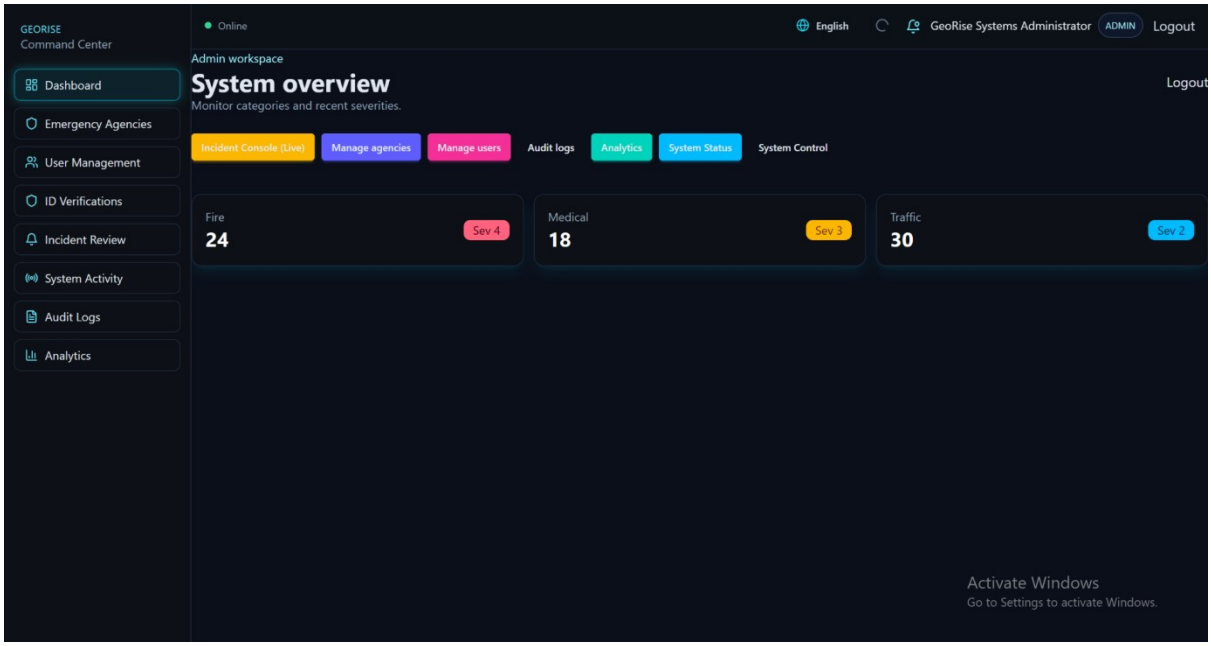

Figure A.9: Admin Overview Dashboard

A.4.2 Agency and Boundary Management

Administrators can approve new agency registrations and update jurisdictional boundaries (Sub-city/Woreda) to ensure accurate spatial filtering.

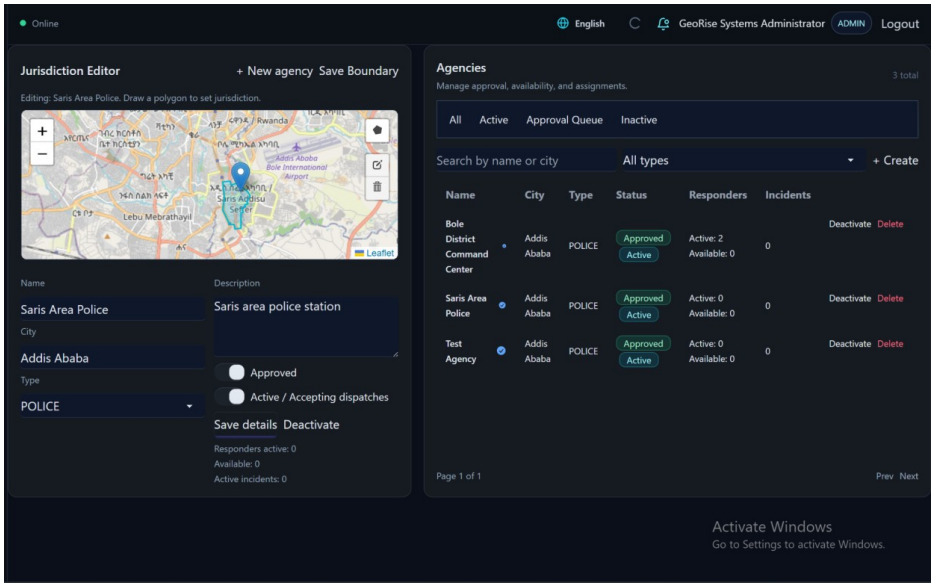

Figure A.10: Agency Management Page

A.4.3 User Verification

To increase the reliability of reports, administrators verify citizen identities by reviewing submitted credentials, which in turn boosts the user's "Trust Score."

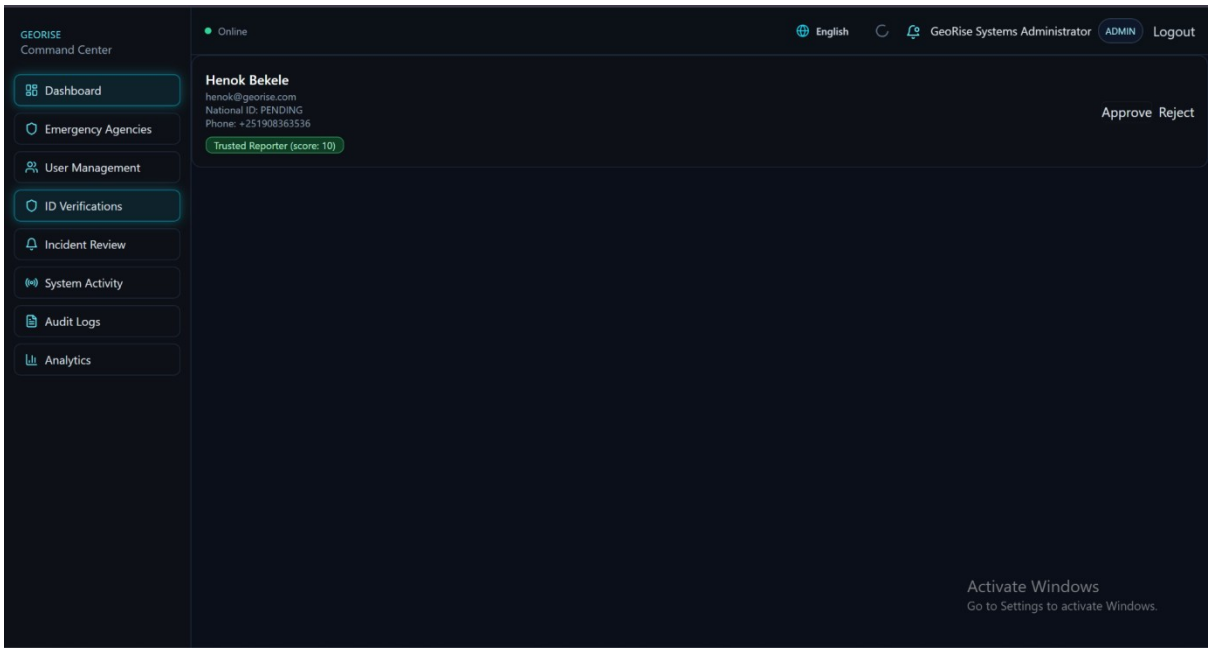

Figure A.11: Citizen Verification Management

B. Data Collection Methods and Tools

B.1 Overview

Data collection for the GEORISE platform was fundamental to establishing a realistic testing framework for AI-driven triage and geospatial jurisdiction enforcement in Addis Ababa. The primary data requirements included precise administrative boundaries (GeoJSON), incident classification keywords in Amharic and English, and a "Golden Dataset" of labeled incidents for model evaluation. By utilizing open-source geospatial repositories and automated geocoding pipelines, the project achieved a high level of geographic fidelity, ensuring that system simulations accurately reflect the urban layout and linguistic context of the city.

B.2 Data Collection Methods

B.2.1 Administrative Boundary Acquisition

To implement jurisdiction-based filtering (e.g., Bole vs. Lideta sub-cities), the project required high-resolution polygon data.

1. **Sourcing**: GeoJSON data representing the Sub-cities and Woredas of Addis Ababa was sourced from the Humanitarian Data Exchange (HDX).
2. **Processing**: The raw coordinates were cleaned and validated using QGIS to ensure "Water-tight" boundaries, preventing spatial overlap during `ST_Within` queries in PostGIS.
3. **Integration**: The final `Ethiopia_AdminBoundaries.geojson` file was used to seed the `Agency` jurisdiction column in the PostgreSQL database.

B.2.2 Multilingual AI Dataset Curation

The AI microservice required a labeled dataset to validate the AfroXLMR classification model.

1. **Keyword Extraction**: Common emergency terms (እሳት/Fire, አደጋ/Accident, ሌባ/Theft) were compiled into a structured `keywords.json` file.
2. **Synthesis**: Using the keywords, a "Golden Dataset" of 50 multi-lingual reports was generated, simulating varied reporting styles (formal vs. colloquial).
3. **Geocoding via OpenCage API**: To ensure realistic spatial distribution, the textual locations in the test dataset were processed through the OpenCage Geocoding API to retrieve exact latitude and longitude coordinates within Addis Ababa.

B.3 Tools and Technologies Used

B.3.1 Software and Libraries

The project leveraged a diverse set of specialized libraries to handle the complex requirements of Natural Language Processing (NLP) and Geographic Information Systems (GIS). For the AI Service Core, the system utilizes a Python-based stack where FastAPI serves as the primary asynchronous framework for building high-performance REST APIs used for model inference. The core linguistic classification is handled by the Hugging Face Transformers library, which is specifically tasked with loading and executing the AfroXLMR-base model to support multilingual text categorization across Amharic and English. Complementing this, NLTK and SpaCy are employed for essential preprocessing tasks such as tokenization and stop-word removal. The reliability of the inference logic is maintained through the Pytest framework, which is used to verify preprocessing heuristics and negation logic.

The backend and data management layer is built on a Node.js stack, utilizing the Prisma ORM to manage relational data modeling and provide an interface for complex raw SQL spatial queries. To ensure system responsiveness during intensive AI computations, BullMQ and Redis are implemented as a distributed task queue, successfully decoupling incident ingestion from high-latency inference tasks. Real-time state synchronization across agency dashboards is achieved through Socket.io, which provides the bi-directional communication layer necessary for pushing incident updates. The backend's robustness is verified through Vitest, which facilitates unit and integration testing of the various services and controllers.

The geospatial and frontend interface relies on Leaflet.js and React-Leaflet as the primary libraries for rendering interactive maps and capturing precise GPS coordinates from the user. On the database side, PostGIS extends PostgreSQL to provide critical spatial functions, enabling operations such as ST_Within for jurisdiction enforcement and ST_DWithin for duplicate detection. For data preparation, QGIS is utilized to clean and validate the GeoJSON administrative boundaries of Addis Ababa before they are seeded into the production database.

B.3.2 External Services

The platform integrates with the OpenCage API as its primary geocoding service to transform textual location descriptions into geographic coordinates. Administrative datasets were sourced from the Humanitarian Data Exchange (HDX) to ensure the accuracy of the jurisdictional boundaries used for spatial filtering.

B.4 Applications and Uses in the Project

B.4.1 Validation of Spatial Queries

The collected GeoJSON boundaries were used to test the `IncidentService.getIncidents` logic. By placing simulated incidents at specific coordinates, the team verified that the PostGIS `ST_Within` function correctly assigned reports to the appropriate agency jurisdiction.

B.4.2 AI Model Benchmarking

The `golden_multilingual.csv` dataset served as the ground truth for evaluating the FastAPI AI service. This allowed for the calculation of Precision and Recall metrics, ensuring the system could reliably distinguish between medical emergencies and fire hazards in both Amharic and English.

B.4.3 Frontend Visualization and Hotspot Analysis

Realistic coordinate data allowed for the testing of the `ClusterLayer` and `HeatLayer` in the Agency Map. This ensured that the frontend could handle high-density incident clusters in areas like Piazza and Bole, providing dispatchers with meaningful situational awareness.

B.5 Sample Data and Outputs

To validate the model's performance across the diverse linguistic landscape of Addis Ababa, a "Golden Dataset" was curated. This dataset provided the foundation for model testing, heuristic development, and geospatial verification.

B.5.1 Input Dataset Structure (incidents_labeled.csv)

The structure of the curated "Golden Dataset" used for model testing and heuristic development is detailed in Table B.1 below.

Table B.1: Sample Structure of AI Golden Dataset (incidents_labeled.csv)

Description	Language	Label
ሰፈር ውስጥ እሳት ተነስቷል	Amharic	FIRE
Car accident near Meskel Square	English	ACCIDENT
Medical emergency, unconscious person	English	MEDICAL
እሳት የለም፣ ግን ጭስ ይታያል (No fire, but smoke)	Amharic	FIRE

B.5.2 AI Classification Output (JSON Response)

The typical structured output generated by the AI Analysis microservice after processing the descriptions from Table B.1 is exemplified in the JSON snippet below.

{

"category": "FIRE",

"severity": 0.85,

"confidence": 0.92,

"heuristics_triggered": ["fire_keyword_am"],

"duplicate_detected": false

}

B.5.3 Geospatial Query Result (PostGIS)

The resolution of specific point coordinates to administrative sub-city jurisdictions via PostGIS spatial operations is presented in Table B.2.

Table B.2: Sample Geospatial Jurisdiction Resolution Results

Longitude	Latitude	Resolved Jurisdiction (Sub-city)
38.7695	9.0034	Bole
38.7521	9.0345	Arada

B.6 Conclusion

The data collection methodology provided the empirical foundation for GEORISE. By combining official administrative data with curated linguistic datasets and robust software libraries, the project successfully moved from a theoretical model to a validated system capable of handling the unique geospatial and cultural demands of Addis Ababa's emergency response infrastructure.